

A Project Report  
On  
**VLM-Assisted Object-Goal Navigation with YOLO and Nav2 for Indoor Mobile Robots**  
BY  
**Mayank Shukla**  
**SE24MROB005**  
Under the supervision of  
**Deep Seth**  
**SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR**  
**DEGREE OF MASTER OF TECHNOLOGY**  
**MT5301: PROJECT TYPE COURSE**



**ÉCOLE CENTRALE SCHOOL OF ENGINEERING**  
**MAHINDRA UNIVERSITY**  
**HYDERABAD**  
**May 2026**

## ABSTRACT

---

Mobile robots navigating both homes and office spaces require not only navigation to waypoint goals but also navigation to objects identified in free-form natural language descriptions, for example, “the grey bottle on the table”. State-of-the-art classical navigation approaches leveraging Nav2 stack and 2D/3D sensory data ensure robust obstacle avoidance and pathfinding capabilities but lack semantic grounding and can only work with predetermined goal coordinates. Moreover, visual object detectors like YOLO enable fast perception of scene semantics but rely upon a limited closed set of class labels and fail to match object descriptions to their visual counterparts.

The current research addresses a feasible approach for designing an architecture which enhances a YOLO + Nav2 framework by incorporating a light-weight VLM as a semantic assistant component. The proposed approach utilizes either on-device YOLOv8 or OAK-D SpatialDetection pipelines to perform object detection for the purpose of getting 2D or 3D object proposals respectively. On the other hand, a light-weight VLM running on an external computing machine analyzes the image patches along with the user's input textual description for deciding if the candidate area contains the desired object, as well as for providing a rough estimate about the direction of the target object.

Differential drive robot experiments on indoor scenarios evaluate the VLM-enabled approach versus a YOLO-only method. The results show that the verification process may help minimize errors due to wrong target identification and boost success rates in open vocabulary commands while still achieving real-time speeds via restricting VLM usage and relying on the existing classical SLAM solution. In indoor experiments on chair, bottle and person targets, the VLM-assisted system increased chair success from 60% to 90%, maintained 90% success for person, and reduced bottle success from 70% to 50%, while increasing detection time due to semantic verification and network delays.

## CONTENTS

---

|                                             |    |
|---------------------------------------------|----|
| Title page.....                             | 1  |
| Abstract.....                               | 2  |
| Chapter 1: Introduction.....                | 4  |
| Chapter 2: Problem Definition.....          | 8  |
| Chapter 3: Background and Related work..... | 11 |
| Chapter 4: Implementation .....             | 17 |
| Chapter 5: Results and Discussion.....      | 42 |
| Chapter 6: Conclusion.....                  | 51 |
| Chapter 7: Reference.....                   | 53 |
| Appendix A.....                             | 54 |

# Chapter 1: Introduction

## 1.1 Motivation

Mobile robots are increasingly deployed in indoor environments like apartments, homes, offices, hospitals and warehouses to perform day to day tasks like inspection, assistance and delivery. These robots can genuinely be useful when they are able to move safely through cluttered spaces and interact with the objects that are relevant to user's task or request. Conventional navigation system focus mainly on geometric planning and obstacle avoidance, but modern applications require robots to have the understanding of what is present in the environment, not just where they can move.

A practical or common use case in object-goal navigation could be that: a user asks the robot to “bring the sanitizer bottle”, “go to the desk with the laptop” or “check for a fire extinguisher”. In this case, the robot would require skills to understanding the language of the user, understanding the scene using its sensors, and then finding the appropriate path to reach the right location. It is important for robots when they have to work in an environment in which objects keep changing their positions and locations from time to time.

At the same time, advancements in research in deep learning have made robust visual objects detectors and foundation models more accessible. Algorithms like YOLO (you look only once) are useful to get fast and reasonably accurate bounding boxes for set number of object categories, while navigation frameworks like Navigation2 (Nav2) stack in ROS2 provide optimal solutions for local and global motion planning. However, combining these conventional algorithms with newer vision-language-models (VLMs) which are capable of reasoning over images and text can open new path ways to more flexible and intuitive human-robot interaction.

## 1.2 Problem Context

In many systems, object-goal navigation is predicted using a predefined catalogue of objects and fixed waypoints. For instance, a robot may be programmed to navigate to “charging dock” or “table” using hard coded coordinates, or it may be restricted to a small set of labels corresponding to classes known to the object detector. This approach works well in controlled environments, but this approach is not scalable when users wants to refer to the objects more naturally, for example, using natural language to describe the objects or when new objects are introduced that are not part of the original training data set.

Pure geometric navigation does not capture the semantics of the environment. A navigation planner can only avoid obstacles and reach a goal pose, but it does not know whether the goal pose corresponds to “next to the chair”, “near the cupboard” or “next to the HOD's cabin”. There is a gap between how humans specify things by giving natural commands and how the present robots interpret and excute the commands given to them.

Vision Language models can be one way to bridge this gap. Given an image and textual description, a VLM can answer the queries about the scene, such as whether the described object is present or not, if present where it is located in the image. If this capability is integrated with the robot's existing preception and navigation architecture, the robot will be able to use high-level semantics to guide its search, rather than relying geometric coordinates and fixed labels.

## 1.3 Limitations of Classical Approaches

Conventional navigation and object detection pipelines have two key limitations:

### 1. Closed vocabulary perception

Models like YOLO are trained on fixed catalogue of classes. They are incapable of recognizing novel objects that weren't present in the training data set nor are they have the ability to handle descriptive queries like “the black box near the shelf” or “the grey bottle on the table”. Sometimes even when the object belongs to the known class, the user may refer to it in language that does not exactly matches the detector's label.

### 2. Weak language grounding

Traditional navigation systems accept goal as coordinates or as reference to known map locations. They are incapable of natively handle natural language commands. Any mapping from user language to the robot must be done as a separate layer, in many this mapping is fragile and relies on fixed template rather than robust semantic understanding.

Because of these limitations the human robot interaction is limited to fixed behaviours or rigid interfaces such as HMI touchscreens and menus. This restricts their uses in unstructured environments, where richer communication and open vocabulary object understanding is desirable.

## 1.4 Role of Vision-Language Models

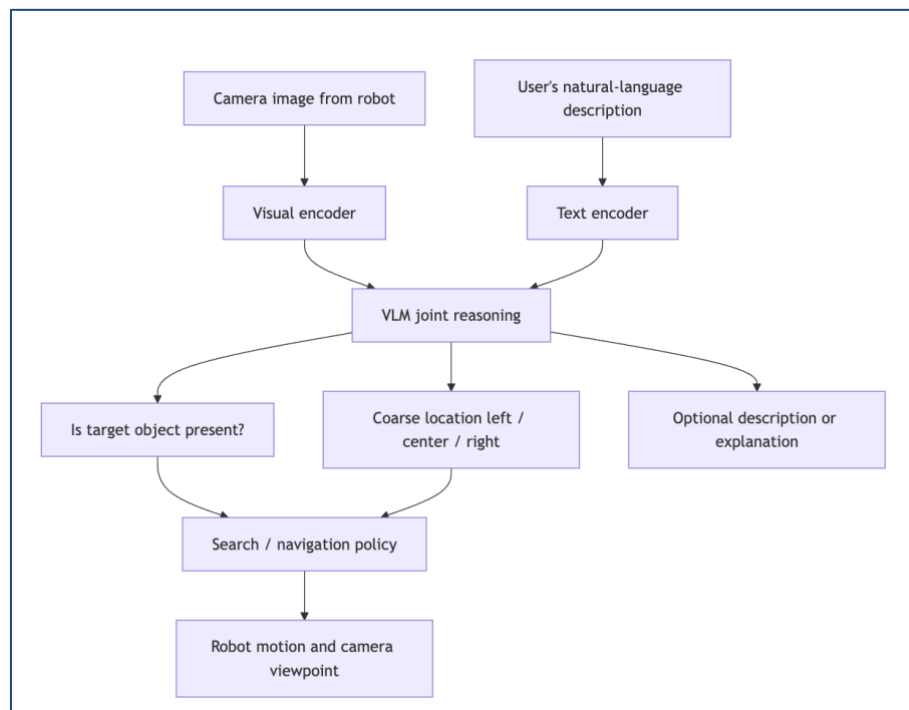


Fig 1.1 VLM working principle

Vision language models are class of models that jointly process images and texts. Given an input image and textual prompt, a VLM can generate description, provide information about the relation between the object in the scene or answer questions. Moreover, VLMs are trained with open-

vocabulary supervision, which helps them to generalize a wide variety of objects and free-form descriptions (fig 1.1).

If integrated with a mobile robot with depth camera and other sensors like 2D lidar, a vlm can be used as a semantic layer above the classical perception and planning layer. For instance, the robot can capture the image feed from its camera and query the VLM with user's natural language based textural description of the target object, and obtain information about whether the desired object is present or not and if present where approximately it appears in the image. This semantic feedback is useful to bias robot's exploration, choose scan directions and validate directions by a standard object detector.

However integrating VLMs into real robots is challenging as many existing works assume powerful GPUs, cloud based computation or simulation environments, but in reality robots operate with limited compute and unreliable network connectivity. Also running large VLM models at high frequency introduces significant latency into control loop, which is undesirable for safe navigation.

## **1.4 Research Gap**

Classical mobile robot navigation stacks such as Nav2 provide reliable geometric planning and obstacle avoidance, but they operate on fixed goal poses or pre-labelled locations and cannot directly interpret natural-language commands about arbitrary objects. At the other extreme, recent vision-language navigation systems often rely on large foundation models or dense visual-language maps, which demand substantial onboard compute, memory and energy and are rarely deployed on resource-constrained robots. These approaches either ignore semantic understanding or assume hardware and mapping pipelines that are difficult to reproduce on an embedded platform. This creates a gap for lightweight, deployable architectures that can accept open-vocabulary object descriptions, use VLMs as semantic helpers, and still remain compatible with standard YOLO + Nav2 navigation on modest hardware.

## **1.5 Objective of This Thesis**

The objective of this thesis is to design, implement and evaluate a mobile robot navigation system that integrates a classical perception and planning algorithm with a lightweight vision language model acting as a semantic helper. The robot uses YOLO for fast bounding box predictions and the ROS2 Nav2 stack for local and global motion planning. Above this architecture a small VLM is integrated to interpret user provided language description and camera images to provide high level cues for object search.

The idea of this proposed architecture is not to replace the existing navigation stack with an end to end model, but to augment it. The VLM provides answer to questions like "is the described object visible?" or "Is it on the left or right side of current view?", the resulting information is used to bolster the search as to where and how the robot searches within an indoor environment. By focusing on resource constrained, real world setup, the work aims to demonstrate that meaningful semantic navigation behaviour can be achieved even when the VLMs runs as an external service with limited capacity.

## 1.6 Contributions

The main contribution of the thesis can be summarised as follows:

- Lightweight VLM semantic verification for object-goal navigation. The system integrates a small vision–language model as a ROI-based semantic filter on top of classical detectors, querying the VLM only on image crops around candidate bounding boxes to decide whether they match the user’s natural-language description.
- Resource-aware hybrid architecture with Nav2. A modular architecture is proposed in which YOLO or OAK-D SpatialDetection provide geometric object proposals, Nav2 handles local and global motion planning, and the VLM runs as an external service with a cooldown mechanism that respects the limited compute and network bandwidth of the robot platform.
- Sector-based search policy guided by semantic cues. The thesis formalizes and implements a search state machine that combines detector outputs, VLM presence and coarse left/center/right location cues, and the robot’s pose to choose scan directions and publish short-range goal poses for Nav2.
- Experimental evaluation on a real robot. The integrated system is evaluated on a differential-drive robot in indoor environments, comparing a YOLO-only baseline and the proposed VLM-assisted configuration in terms of task outcome, detection time and qualitative semantic correctness.

These contributions aim to progress towards more intuitive and flexible human robot interaction, while remaining grounded in the practical realities of embedded computation and existing navigation frameworks.

## Chapter 2: Problem Definition

The aim of this work is to enable mobile robot platform to perform object-goal navigation using both classical navigation and perception stacks and a lightweight vision language model in indoor environments. The robot should be able to receive the tasks as natural language description of the object, search for the object using the sensors onboard and navigate towards them safely.

### 2.1 Basic Questions Addressed

The work is designed to answer the following question:

**1. Language to behaviour:**

Given a natural user command like “find the red bottle kept on the center table”, can the robot interpret the language and convert it into a search behaviour in an indoor environment?

**2. VLM-assisted-perception:**

Whether a small vision language model, running as an external service, reliably indicate whether the described object is present in the robot’s camera view and provide location information that can assist the object search ?

**3. Integration with classical navigation:**

How can semantic information provided by the VLM be combined with detection from a YOLO based object detector and with the existing Nav2 navigation stack so that overall system remains safe and responsive while benefitting from language guided cues ?

**4. Effects on navigation performance:**

In indoor scenarios, does the integration of VLM based semantic guidance improves the robot’s ability to find user specified objects compared to a baseline system that uses only YOLO and Nav2 without any VLM component?

These questions are experimentally evaluated on three object categories (chair, bottle, person) using a YOLO-only baseline and the proposed VLM-assisted system.

### 2.2 Formal Problem Statement

Consider an indoor environment represented by a continuous state space of robot poses  $\mathcal{X} \subset \mathbb{R}^3$ , where a pose  $x \in \mathcal{X}$  represents the robot’s position and orientation. At each time step  $t$ , the robot observes the environment through its sensors and receives an RGB image  $I_t$  from its depth camera. The user commands an object-goal navigation task using natural language description  $q$  (for example, “the kettle in the kitchen”).

The robot has:

- An object detector  $D$  (YOLO) that can map an image  $I_t$  to a catalogue of bounding boxes and class labels.



- A vision language model  $V$  that, given an image  $I_t$  and text command query  $q$ , can produce semantic information about whether the described object is present and if so its location in the image.
- A navigation stack  $P$  (Nav2) that computes collision free path to the goal  $x_g$  from the current pose  $x_t$ .

The entire problem can be expressed as designing a policy  $\Pi$  that at each step selects the next navigation action based on the robot's state, sensor data and query:

$$\alpha_t = \Pi(x_t, I_t, q, h_t)$$

Where  $h_t$  encapsulates all relevant historical information like previous detection and VLM actions. The action  $\alpha_t$  is performed by the low level controller using the Nav2 stack, leading to a new pose  $x_{t+1}$ .

The aim of the policy is to guide the robot from its initial state  $x_0$  to the goal region  $G(q)$  which means the robot is close enough to an instance of the object represented by  $q$ , without any collision and while considering the kinematics of the robot. In other words, the robot should terminate its motion when:

- the target object is recognized in the environment with high confidence by the perception module, and
- the robot is at an acceptable distance  $d_{goal}$ .

## 2.3 Role of the Vision Language Model

In this framework, the vision-language model does not perform low-level control tasks. It rather performs the semantic task:

$$S_t = V(I_t, q)$$

where  $S_t$  represents high-level knowledge, like:

- whether there is an object corresponding to the description in the current view, expressed in binary format; and
- the coarse location of the object in the view (left, center or right).

The semantic output is then integrated with YOLO detection results and the robot's pose estimation to form the policy  $\Pi$ . For instance, if  $S_t$  keeps pointing out that the target object may be present at the left of the scene, the robot can direct its search and movements towards the left before depending on YOLO to detect the target object accurately.

## 2.4 Scope and Assumptions

In order to reduce the complexity of the task, the following assumptions are made:

- The environment consists of an indoor setting with relatively static obstacles and few moving objects.

- The robot has good odometry and simple localization capabilities for short-term navigation using the Nav2 stack.
- The vision-language model is run on a separate machine that is connected to the robot via a wireless network and called at a frequency lower than the camera's framerate.
- The main concern is to show that even with such a lightweight implementation of the vision-language model, it can serve its purpose of guiding the robot in object-goal navigation, rather than being able to achieve optimal results or perform open world tasks.

The work in this thesis focuses specifically on single-object, indoor object-goal navigation in relatively small office-like environments. The robot receives one natural-language description  $q$  at a time, searches for a single matching object instance, and navigates to a safe standoff pose without collision using the Nav2 stack. Long-horizon task planning, multi-step instructions, multi-robot coordination, and lifelong semantic mapping are outside the scope of this work. The objective is to demonstrate that even under these constrained conditions, a lightweight VLM used as a semantic verifier can meaningfully improve object-goal navigation on a resource-limited platform.

## **Chapter 3: Background and Related work**

### **3.1 Classical Mobile Robot Navigation**

Mobile robot navigation is composed of mapping, localisation, perception, path planning and control. A robot uses depth and range data from the sensor to estimate free, occupied and unknown spaces in indoor environments, then computes a collision free path from its current pose to a desired goal pose using a global planner, while the local planner is used to detect nearby obstacle and dynamic corrections during execution. This navigation framework is a standard because it is modular reliable and efficient on real-world robot.

However, classical navigation primarily operate on geometry rather than semantics. They respond to goal pose of location and answers whether the location is reachable, but they don't respond to semantic description of the goal like “go towards the black chair” or “go near the emergency exit door” without an additional semantic reasoning layer. This limitation becomes important in human robot interaction scenarios where users naturally specify the goal in language instead of cartesian coordinates or pre-labelled map locations.

### **3.2 Object-goal navigation and open-vocabulary search**

To progress beyond Cartesian coordinate-based navigation, researchers have introduced object-goal navigation, in which the robot navigates to a semantic object category rather than to a fixed waypoint. In this setup, the robot is asked to search for an instance of an object such as a bed, table or chair using its sensory observations and prior knowledge of indoor environments. Early object-goal navigation methods relied on object detectors with a closed catalogue of classes and semantic segmentation models, meaning the robot could only search for object types that were present in the training data.

This closed-vocabulary framework is restrictive in real environments, where a user may describe an object that is not part of the detector's label set or may refer to it using flexible natural language. Open-vocabulary object-goal navigation therefore has become an important research direction, where the target is specified with natural language and the robot must generalise beyond a fixed class list. HM3D-OVON provides a dataset and benchmark for this setup, defining open-vocabulary object-goal navigation in expansive, realistic 3D environments and testing agents that must generalise to object types they have not encountered during training.

DivScene further investigates open-vocabulary object-goal navigation using large vision-language models (LVLMs) as controllers in a variety of indoor scenes. The authors show that naively using LVLMs to drive the entire navigation policy is still far from robust and requires task-specific adaptation and fine-tuning, particularly in cluttered or visually complex environments. These works are important for the present thesis because they expose both the potential of open-vocabulary ObjectNav and the practical difficulties of deploying LVLM-based agents.

**Table 3.1: summarises representative open-vocabulary ObjectNav and VLM-based navigation approaches and contrasts them with the system proposed in this thesis.**

| Method / Paper                                  | VLM / model type                                    | Dataset / environment               | Semantic mapping                                          | Real-robot demo                          | Compute requirement                                                            | Main weakness                                                                                 |
|-------------------------------------------------|-----------------------------------------------------|-------------------------------------|-----------------------------------------------------------|------------------------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| HM3D-O VON (Bajpai et al.)                      | Open-vocabulary detector + CLIP-style VLM           | HM3D (simulated homes)              | Yes – semantic occupancy map                              | No (simulation only)                     | High GPU for dense mapping and policy learning                                 | Heavy mapping pipeline; difficult to deploy on embedded robot; no real-world latency analysis |
| VLMaps (Huang et al.)                           | CLIP visual-language encoder                        | ScanNet / indoor scenes             | Yes – dense visual-language 3D map                        | Limited (mainly mapping demos)           | High memory and compute for storing per-pixel embeddings                       | Requires dense map building; not suitable for low-compute, online navigation on small robots  |
| LVLM-based nav (e.g. recent large-VLM pipeline) | Large vision-language model (cloud or high-end GPU) | Custom simulated + some real scenes | Often no explicit map; uses language-conditioned policies | Sometimes; usually with base station GPU | Strong dependence on powerful GPU or cloud; high network and inference latency | Hard to run on small platforms; limited analysis of resource usage and failure modes          |
| Classical YOLO + Nav2 (baseline)                | Closed-set object detector only                     | Real indoor labs / offices          | No semantic map; geometric costmap only                   | Yes                                      | Embedded GPU or CPU for detector, moderate                                     | Cannot handle open-vocabulary queries; no language grounding                                  |

| Method / Paper                                  | VLM / model type                               | Dataset / environment                        | Semantic mapping                                                   | Real-robot demo                | Compute requirement                                                          | Main weakness                                                                                                 |
|-------------------------------------------------|------------------------------------------------|----------------------------------------------|--------------------------------------------------------------------|--------------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
|                                                 |                                                |                                              |                                                                    |                                |                                                                              | beyond class labels                                                                                           |
| <b>This thesis (YOLO/OAK + lightweight VLM)</b> | Small external VLM with ROI-based verification | Real indoor lab; chair, bottle, person tasks | No dense map; uses ROI semantics and coarse left/center/right cues | Yes (differential-drive robot) | Embedded robot + external CPU/GPU over Wi-Fi; query rate limited by cooldown | Limited to single-object tasks; small objects (bottles) remain challenging; added latency due to external VLM |

The works in **Table 3.1** highlight a clear trade-off between semantic richness and deployability. Large-scale benchmarks such as HM3D-OVON and DivScene demonstrate that LVLM-based agents can tackle open-vocabulary ObjectNav in simulation, but they typically assume powerful GPUs and do not address real-world constraints such as latency, bandwidth and on-board compute. Map-centric approaches like VLMaps and uncertainty-aware frameworks explicitly model semantic information and its uncertainty over time, yet require dense CLIP embeddings and probabilistic geometric–semantic maps that are challenging to maintain on embedded platforms. Recent VLM-assisted navigation systems show that language models can act as high-level advisors on real robots, but they focus on social navigation or path-blocking scenarios rather than object-goal search on constrained hardware. In contrast, the present work deliberately forgoes dense semantic maps and large LVLM controllers in favour of a small VLM used purely for ROI-based semantic verification on top of a standard YOLO/OAK + Nav2 pipeline, targeting practical deployment on a resource-limited indoor robot.

### 3.3 Visual Language maps and semantic memory

Moving from purely geometric maps to semantic maps that store language-aligned visual information is the goal of one prominent line of work. Huang et al. propose Visual Language Maps (VLMaps), which incorporate visual–language features into a 3D reconstruction of the environment so that text queries and images share a common embedding space. During exploration, the robot inserts dense image features from a pretrained vision–language model into the map, enabling queries such as “between the sofa and the TV” to be grounded directly in space.

The main advantage of VLMaps is that, instead of relying solely on what is visible in the current frame, they provide a semantic memory for language-guided navigation. This memory allows the

robot to ground new language instructions by recalling previously observed parts of the environment, rather than searching only based on the latest camera view. This idea is highly relevant to the present thesis because it shows that vision–language models can contribute not only to captioning or question answering, but also to the robot’s spatial representation.

Formally, VLMaps compute dense visual–language embeddings  $\mathbf{f}_i \in \mathbf{R}^d$  for each image pixel using a CLIP-style encoder and back-project these features into a 3D map of the environment. A text query  $\mathbf{q}$  is encoded into a vector  $\mathbf{g}(\mathbf{q})$ , and semantic relevance at each map cell is obtained via cosine similarity

$$S_i = \frac{\mathbf{f}_i \cdot \mathbf{g}(\mathbf{q})}{\|\mathbf{f}_i\| \cdot \|\mathbf{g}(\mathbf{q})\|},$$

so that locations with high similarity scores can be directly selected as navigation targets. This mathematically grounded use of embeddings motivates the present work’s use of VLM outputs as semantic signals, even though we avoid constructing a full VLMap due to memory and compute constraints on the robot platform.

There are, however, important deployment limitations. Constructing and maintaining a dense language-aligned map requires frequent feature extraction, 3D reconstruction and storage of high-dimensional embeddings, which is challenging to operate in real time on a mobile robot with limited CPU, GPU and potentially intermittent connectivity. Therefore, the current work takes inspiration from the semantic guidance concepts of VLMaps but deliberately avoids building a dense semantic map, instead adopting a lighter integration that builds upon an existing YOLO + Nav2 pipeline and utilises a small VLM as a semantic helper.

### 3.4 Uncertainty-aware active perception

The notion that vision-language outputs should be viewed as uncertain observations that can direct investigation rather than just as right or wrong is another significant advancement. For open-vocabulary object-goal navigation, Bajpai et al. present an uncertainty-informed active perception framework in which the robot maintains a semantic belief over the environment and selects actions that lessen ambiguity about the location of the target item. Instead of using a purely greedy or shortest-path search, their approach blends open-vocabulary object reasoning with an exploration policy that actively chooses useful viewpoints.

This contribution highlights an issue in VLM-assisted robotics: semantic predictions are often, noisy or viewpoint-dependent, especially for small or partially occluded objects . By explicitly modelling uncertainty, the robot can avoid over-committing to weak evidence and instead move in ways that gather more informative observations . Conceptually, this is similar to the long-term research direction of building a semantic memory and uncertainty-aware exploration framework for open-vocabulary navigation.

However, the full uncertainty-aware formulation is also more complex than the scope of many practical systems. It typically requires a probabilistic geometric–semantic map, an uncertainty model for perception and a planner that optimizes information gain or a related objective . While this is a good research direction, it is computationally heavier than a lightweight YOLO + Nav2 +

helper-VLM integration. Due to this reason, the current work focuses on a simpler semantic assistance strategy without incorporating a complete uncertainty-aware planner, while still addressing this as an important avenue for future work.

### 3.5 Language- and VLM-assisted navigation systems

Beyond semantic maps and active perception, recent work has explored using large language models (LLMs) or vision–language models (VLMs) as task-level advisors for robot navigation. In these systems, the model doesn’t replace the low-level controller; instead, they help to interpret natural-language instructions, select subgoals, reason about environment context or decide which perception tools to invoke. Xue et al. present such an approach in “Integrating Large Language Models for Intuitive Robot Navigation”, showing how LLMs can be integrated into navigation architectures to make task execution more intuitive for users while keeping classical navigation modules in the loop .

A closely related and more recent example is “Unblocking the Path: VLM-Assisted Robot Navigation in Indoor Environments” . That work investigates how a VLM can assist a robot in reasoning about obstacles and path-blocking situations that are challenging for purely geometric planners . The VLM is used as a semantic problem-solving module layered on top of an existing navigation system, helping the robot understand what is blocking its path (for example, doors or movable objects) and how it might resolve the situation . This architecture is conceptually similar to the approach in the present thesis: the VLM is used as a high-level semantic helper while the underlying motion planning and control remain classical , .

These language- and VLM-assisted systems demonstrate a practical compromise between black-box end-to-end learning and purely classical navigation. They illustrate that it is possible to benefit from the semantic flexibility of foundation models while maintaining the reliability and predictability of engineered navigation stacks , . This observation strongly influences the architectural choices made in this work.

### 3.6 Position of the present work

The literature reviewed above can be grouped into three main categories: open-vocabulary object-goal navigation benchmarks and LVLM controllers , , semantic mapping and memory , and uncertainty-aware or language-assisted navigation strategies , , . All of these works move toward more semantically aware and language-capable robots, but they also tend to assume either powerful compute, complex mapping pipelines or large general-purpose models –.

The system developed in this thesis is intentionally positioned as a more practical and resource-aware instance of VLM-assisted object navigation. Instead of building a dense visual language map or implementing a full uncertainty-informed exploration planner , the proposed system integrates a small VLM into an existing YOLO + Nav2 pipeline on a real robot. YOLO provides fast on-board object detection, Nav2 handles geometric planning and control, and the VLM is used only as a semantic helper that interprets user-provided language descriptions and camera images to provide high-level cues for object search. Compared to LVLM-based approaches that train large models as end-to-end navigation policies , , and to agentic LLM systems that handle task-level planning , , this

design prioritizes simplicity, deployability and compatibility with resource-constrained hardware while still enabling open-vocabulary, language-based object navigation.



## Chapter 4: Implementation

### 4.1 System Overview

The proposed system is a differential-drive mobile robot equipped with an OAK-D Lite RGB-D camera and a 2D LiDAR, running ROS 2 Humble on a Jetson Orin Nano. All safety-critical perception, localization and navigation functionality executes on the Jetson, while a lightweight vision-language model (VLM) runs on a separate machine connected over a wireless network. The overall goal is to support object-goal navigation with open-vocabulary target descriptions: a user specifies a target in natural language, and the robot searches for the described object and navigates to a safe standoff pose once it is found.

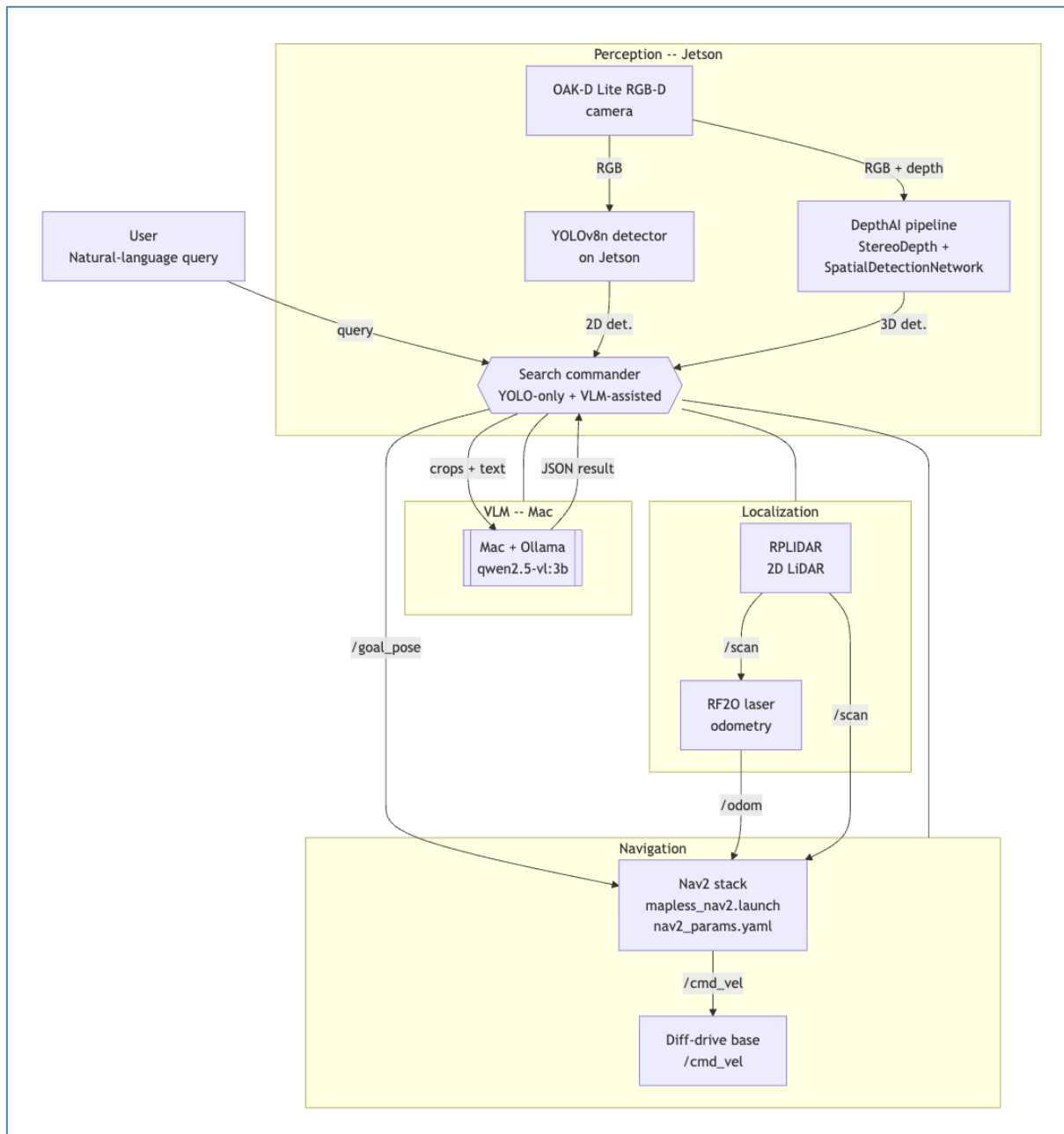


Fig 4.1 Overall system architecture diagram

**Figure 4.1** shows the overall system architecture, highlighting the interplay between perception, semantic reasoning, localization and navigation. At the core of the navigation capability is the ROS 2 Navigation2 (Nav2) stack, launched in a mapless configuration using `mapless_nav2.launch.py` and the `nav2_params.yaml` configuration file. Nav2 provides global planning, local control, behaviour trees and velocity smoothing via a `/goal_pose` interface, which accepts goals in the robot's local frame, and a `/cmd_vel` interface that drives the base through a velocity smoother. Once a semantic target has been identified and converted into a geometric goal, motion is therefore executed using well-tested navigation components rather than ad-hoc control code.

Perception is provided by two sensing pipelines. First, an OAK-D Lite camera delivers synchronised RGB and stereo depth streams through a DepthAI pipeline that runs directly on the device: a StereoDepth node estimates disparity and a SpatialDetectionNetwork running a yolov6-nano model yields 3D spatial coordinates for each detection in the camera frame. Second, a 2D LiDAR is launched using `rplidar_ros` together with RF2O laser odometry, providing planar obstacle information and an odometry estimate that Nav2 uses for local planning and for the closed-loop turning behaviours in the search commanders.

On top of the raw sensing capabilities, the system implements several perception and decision modules. A YOLO-only search commander (`ros2_yolo_vlm.py`) performs class-based object search using an on-board Ultralytics YOLOv8 detector on the RGB stream from the OAK camera: the node interprets the user's query as a target class, detects candidate objects, and combines 2D bounding boxes with depth data to obtain relative goal poses that are sent to Nav2. A separate OAK spatial commander uses the DepthAI SpatialDetectionNetwork to obtain 3D detections from the yolov6-nano model running on the OAK hardware, which reduces CPU load on the Jetson and provides a more classical, hardware-accelerated spatial perception baseline.

The semantic layer is implemented by the YoloVlmCommander node in `vlm_commander.py`, which augments the classical perception pipelines with ROI-based semantic verification from a VLM. This node subscribes to the OAK spatial detections, extracts an RGB crop for each candidate and sends the crop together with the user's natural-language description to a qwen2.5vl:3b model hosted on a separate Mac via an OpenAI-compatible HTTP API provided by Ollama. The VLM returns a JSON response with fields such as `match`, `score`, `reason` and `object_type`, indicating whether the crop likely matches the described object and with what confidence. Only detections whose VLM score exceeds a threshold are accepted; the associated DepthAI spatial coordinates are converted into a Nav2 goal in the `base_link` frame with a safety standoff, and the goal is published on `/goal_pose`. In this design, the VLM acts purely as a semantic filter on top of classical perception and navigation, rather than as a direct controller of robot motion.

From an information-flow perspective, the complete system operates as follows. A user issues a natural-language query specifying the desired object, which is received by either the YOLO-only search commander or the VLM-assisted YoloVlmCommander. The commander controls the OAK pipeline and, for each frame, obtains RGB images and, in the OAK-based configurations, spatial detections from the DepthAI network. In the YOLO-only configuration, detections are directly converted into candidate goal poses and sent to Nav2, which plans and executes motion. In the VLM-assisted configuration, each detection proposal is first passed through the VLM: cropped ROIs and the text query are sent to the remote qwen2.5vl:3b model, which returns a confidence score and explanation; only high-confidence matches are translated into Nav2 goals. In both cases, Nav2 generates safe velocity commands for the base using LiDAR-based obstacle information and odometry, while the higher-level commander determines when and where new goals should be issued based on visual and semantic feedback.

## 4.2 Hardware and Platform

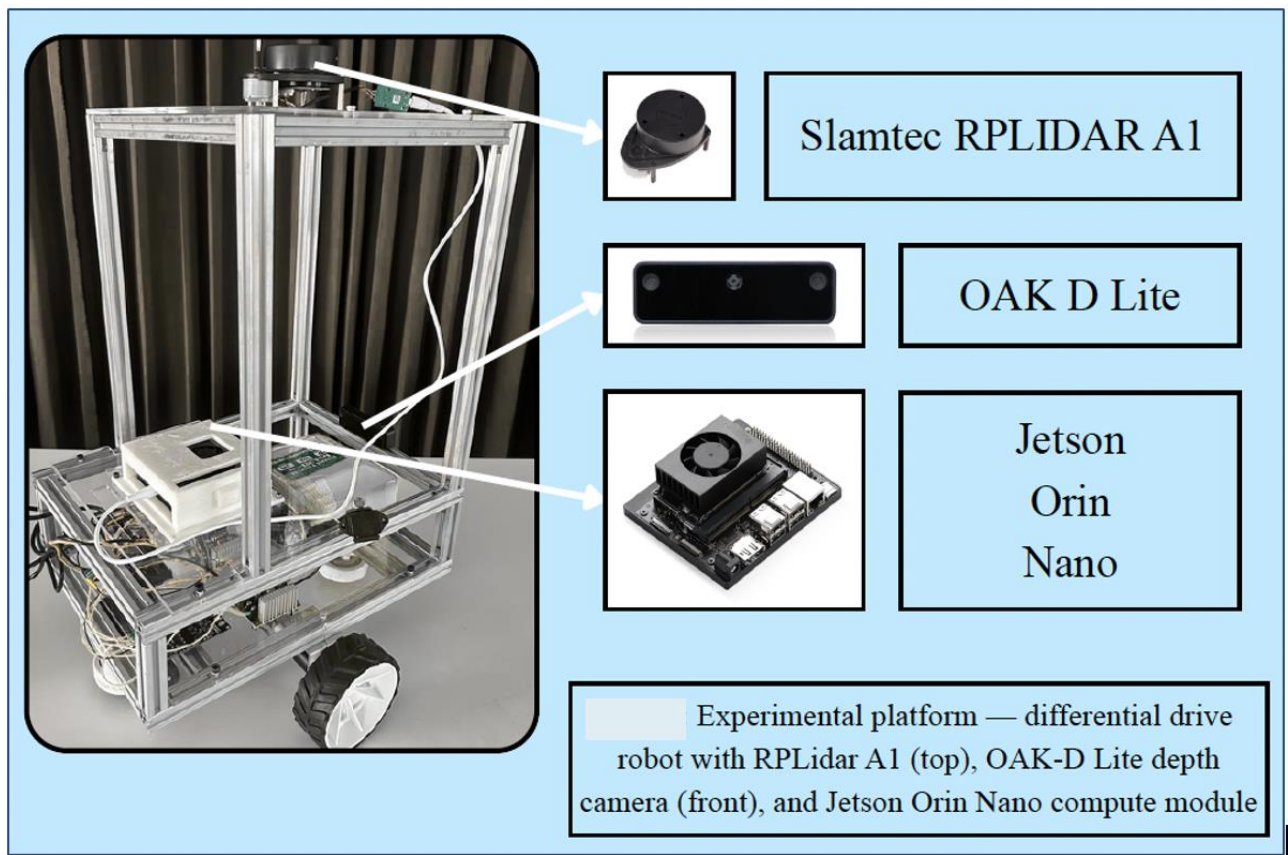


Fig 4.2 Hardware platform

The experimental system consists of a differential drive robot controlled using a typical ROS 2 `/cmd_vel` velocity interface. The low-level operation of wheels is linked using the `noodom_diff_drive_bridge`, and the robot's pose utilized by the navigation and searching parts comes from the `/odom` topic. Currently, the odometry used for the navigation process does not come only from wheel encoders, and instead uses RF2O Laser Odometry, launched via `rf2o_laser_odometry_node` and set up to provide an odometry and a related transform from `odom` to `base_footprint` at a rate of 10 Hz. Such a solution offers a convenient source of localization for indoor mapless navigation tasks, especially those requiring 45° rotation combined with forward movement as in the object search strategy (fig 4.2).

The sensors aboard the robot include a visual OAK D Lite RGB-D camera and a range-finding 2D RPLIDAR laser scanner. For the purposes of perception and semantic reasoning, the OAK D Lite is used as the main visual sensor. Within the ROS bringup configuration, the camera is launched via `depthai_ros_driver`, called `oak`, with its depth aligned with the RGB stream and stereo mode activated. The stereo pipeline parameters are set up for precise depth estimation with subpixel disparity and several filters applied to the depth stream, including spatial, temporal, speckle and median. In addition to launching the robot, the VLM commander initializes a custom DepthAI pipeline, which consists of the OAK color camera on `CAM_A` along with the stereo cameras on `CAM_B` and `CAM_C`; RGB and stereo depth streams are outputted with a resolution of  $640 \times 400$  and a 20 FPS frame rate, respectively. Practically, the OAK camera is installed on the robot in a position where its RGB view and calculated depths cover the search space in front of the robot body.

Planar RPLIDAR scanner is the second sensing modality, launched using *rplidar\_ros* with the frame name *laser\_frame*, at serial baud rate of 115200 and angle compensation enabled. */scan* topic is published by *rplidar* and then used by RF2O and by the navigation stack for short-range obstacle awareness. In addition to the LiDAR, the system derives an auxiliary scan from the OAK depth point cloud using *pointcloud\_to\_laserscan*, generating */oak/scan* after filtering points by height and range. The configuration retains points between 0.10 m and 0.62 m in height and between 0.3 m and 3.0 m in range, which represents frontal obstacles seen by the depth camera while excluding floor and ceiling returns. Together, LiDAR and RGB-D camera provide complementary geometric perception for safe indoor navigation and target search.

Computation is split across two machines. The onboard computer i.e jetson orin nano hosts ROS 2 Humble, the robot bringup stack, Nav2, DepthAI integration and the perception nodes. This computer is responsible for real-time sensing, search state management, goal publication and motion execution. For the semantic layer, a separate Mac machine is used to host the lightweight vision–language model through Ollama, exposing an OpenAI-compatible HTTP endpoint. In the implementation, the *YoloVlmCommander* node connects to this external machine through *base\_url=f"http://{MAC\_IP}:11434/v1"*, where the configured IP address is *172.20.10.7* and the service port is 11434. The model queried by the robot is *qwen2.5vl:3b*, which is invoked only when candidate detections require semantic verification.

The communication between the robot and the external Mac is carried over a Wi-Fi hotspot, which introduces variable network latency and makes it undesirable to call the VLM at camera frame rate. For this reason, the semantic module is designed as an occasional advisor rather than a continuous controller. In *vlm\_commander.py*, repeated queries are rate-limited using the variable *vlm\_cooldown*, which is updated so that at least about 2 seconds elapse between VLM calls in sectors where no sufficiently strong match is found. The OpenAI client itself is also configured with a 20-second timeout, reflecting the fact that remote VLM inference is much slower and less predictable than local geometric perception. This hardware and networking split is an important design choice in the present work: all safety-critical control and navigation remain local to the robot, while the external VLM contributes only high-level semantic judgments when needed (Table 4.1).

**Table 4.1: Hardware components used in the system**

| Component               | Model            | Role in system                                              | Typical rate / FPS | Notes                                  |
|-------------------------|------------------|-------------------------------------------------------------|--------------------|----------------------------------------|
| <b>Onboard computer</b> | Jetson Orin Nano | Runs ROS 2 nodes, Nav2 stack, baseline perception, control  | —                  | Main embedded compute platform         |
| <b>RGB-D camera</b>     | OAK-D Lite       | Captures RGB and depth; used for YOLO/OAK spatial detection | 30 FPS (typical)   | Provides color and depth streams       |
| <b>2D LiDAR</b>         | RPLIDAR          | Obstacle sensing, local environment                         | 10 Hz (typical)    | Used with RF2O / localization pipeline |

|                          |                          |                                                 |                        |                                 |
|--------------------------|--------------------------|-------------------------------------------------|------------------------|---------------------------------|
|                          |                          | structure,<br>odometry aid                      |                        |                                 |
| <b>Mobile robot base</b> | Differential drive base  | Executes velocity commands and navigation goals | Control-rate dependent | Physical robot platform         |
| <b>External VLM host</b> | MacBook / remote machine | Runs lightweight VLM inference service          | Query-based            | Connected over wireless network |

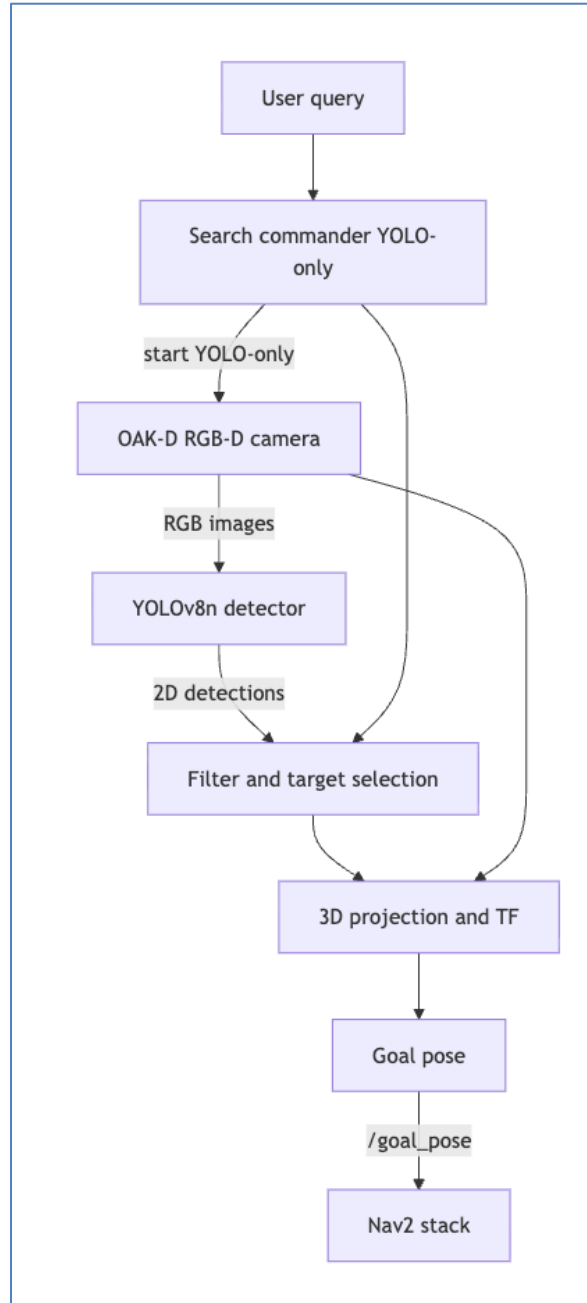
### 4.3 Baseline Object-Goal Navigation with YOLO + Nav2

The first baseline considered in this thesis is a purely YOLO-based object-goal navigation pipeline without any VLM assistance. In this configuration the robot treats the user’s description as a target detector class, runs an Ultralytics YOLOv8 model on RGB images from the OAK-D Lite camera, and converts 2D detections into geometric goals using depth information and the Nav2 stack. This baseline represents a typical closed-vocabulary perception and navigation system against which the benefits of semantic verification can be evaluated.

#### 4.3.1 YOLO-based perception

The baseline perception module uses Ultralytics YOLOv8n as an onboard 2D object detector, initialized in the node through *self.yolo = YOLO('yolov8n.pt')*. This detector operates on RGB images acquired from the OAK camera and is configured with a confidence threshold of 0.35 to suppress weak detections. In the node constructor, the camera topics */oak/rgb/image\_raw* and */oak/stereo/image\_raw* are subscribed using *message\_filters.Subscriber*, and the streams are synchronized through an *ApproximateTimeSynchronizer* with queue size 10 and slop 0.15 s. The synchronized callback stores the latest RGB frame and aligned depth frame, which are then used jointly in the search loop.

When YOLO detects a candidate target object, the 2D bounding box is converted into a metric goal in the robot frame using the *navigate\_to\_target()* function. First, the center pixel of the bounding box is computed as the midpoint of the detected rectangle. Since the RGB and depth images may have different dimensions, the box center is scaled from RGB coordinates into the corresponding depth image coordinates using the ratios between the two frame sizes. The value of depth at the mapped pixel is determined using the 16-bit stereo depth image, and when the value is zero or invalid, the detection is rejected (fig 4.3).



**Fig 4.3 YOLO-only perception pipeline block diagram**

When the depth value is obtained, the node uses a simple back projection method to estimate the relative 3D position of the object. The forward distance is obtained directly as  $z_{forward} = \text{depth\_mm}/1000$ , while the lateral displacement is approximated using the camera focal length estimate  $f_x = \frac{W}{2 \tan(73^\circ/2)}$  and principal point  $c_x = W/2$ , where  $W$  is the depth image width. This gives us a lateral offset determined by the horizontal shift of the depth pixel from the image center. This is subsequently transformed to the robot-centric coordinate system, represented by  $raw\_x$  (forward) and  $raw\_y$  (lateral). Using these two distances, the distance to the object is calculated. Rather than driving straight ahead towards the object, a 0.5-m standoff distance is enforced. To further improve safety and stability, the translational step toward the object is limited by  $self.max\_goal\_step = 2.0$ , so the published Nav2 goal remains local even when the object is

detected farther away. The resulting target position and orientation are packaged as a *PoseStamped* message in the *base\_link* frame and sent to */goal\_pose* for execution by Nav2. The exploration and goal-selection logic built on top of this perception module is described in Section 4.3.3.

### 4.3.2 Language parsing for open-vocabulary queries

Although this baseline does not use a VLM, it still supports a simple form of language-based target specification through the *parse\_query()* function. This function receives the user’s natural-language command, converts it to lowercase and removes punctuation using a regular expression so that only alphanumeric tokens remain. A set of common stopwords such as “find”, “the”, “go”, “towards”, “search” and “detect” is then filtered out, leaving only potentially meaningful target words.

The next step is to map the user’s wording into one of the fixed YOLO classes from the COCO label set stored in *COCO\_LABELS*. The function first checks whether any full label already appears in the normalized text. If not, it consults *SYNONYM\_MAP*, which resolves common user expressions to detector labels, for example *sofa*  $\rightarrow$  *couch*, *table*  $\rightarrow$  *dining table*, *desk*  $\rightarrow$  *dining table*, *phone*  $\rightarrow$  *cell phone*, *plant*  $\rightarrow$  *potted plant*, and *computer*  $\rightarrow$  *laptop*. This improves upon a system that strictly uses label exact interface, since the user is no longer required to know the exact detector language beforehand.

After selecting the target class, the filtered tokens which are not part of the label itself or a synonym for that label are saved under *target\_attributes*. For example, a query like “Find me the red bottle” will classify the object class as bottle, while retaining red as an attribute token. However, currently in this baseline approach, these attribute tokens are read and saved but do not play any role in visual reasoning since YOLOv8n can only predict closed set object classes but not semantic descriptions such as colors or context. It is exactly this reason that makes our baseline approach known as a limited open vocabulary layer; although there is freedom in natural language queries, the grounding capability still relies on *COCO* object classes.

### 4.3.3 Exploratory search behaviour

Moreover, the baseline robot also utilizes an active search process such that it is not dependent entirely on having the target within range from the very beginning. The state machine used for this purpose has been defined within the function *control\_loop()* where initially the robot state is set to *STATIONARY\_SCAN*. While in this state, the robot stays still and performs continuous YOLO on the current RGB image for a certain period, *self.scan\_duration* = 5.0. If a valid detection of the parsed target class is produced during this interval, the robot halts scanning and immediately computes a navigation goal toward the target.

If no target is detected within the scan interval, the robot initiates a 45° turn to inspect the next sector. This turning behavior is closed loop and is based on the robot yaw obtained from the */odom* topic. In *start\_turn\_45()*, a new *target\_yaw* is computed by adding 45° to the current yaw and wrapping the result into the  $[-\pi, \pi]$  range. During subsequent control iterations, the angular error between *target\_yaw* and *current\_yaw* is calculated, again wrapped to the principal range, and a proportional controller commands *cmd\_vel.angular.z* until the error falls below 0.05 rad, or about 2.8°. The angular speed is limited between a minimum of 0.15 rad/s and the configured *self.angular\_speed* = 0.5 rad/s so that the robot slows down as it approaches the target heading. Once the turn is finished, the robot briefly pauses, returns to the *STATIONARY\_SCAN* state, and starts scanning the new sector.

The process of searching is organized in sectors by setting *self.max\_turns\_before\_move* = 8, meaning that there will be a full turn around the robot with 45° intervals. In case the robot manages to scan eight sectors and fails to find the required object, it triggers *move\_to\_new\_vantage\_point()*, publishing the command to move the robot forward 1.0 m in *base\_link* frame and changing the robot's mode to *WAIT\_FOR\_MOVE*. After a certain delay, the robot stops and starts scanning again using the stationary scanning procedure. Thus, the algorithm implements a fairly effective search strategy: scan nearby, turn completely, and then move to a new position. The same algorithm will be applied later in another commander, namely, in VLM assisted commander, and can serve as a benchmark for analyzing the value added by semantic verification (fig 4.4).

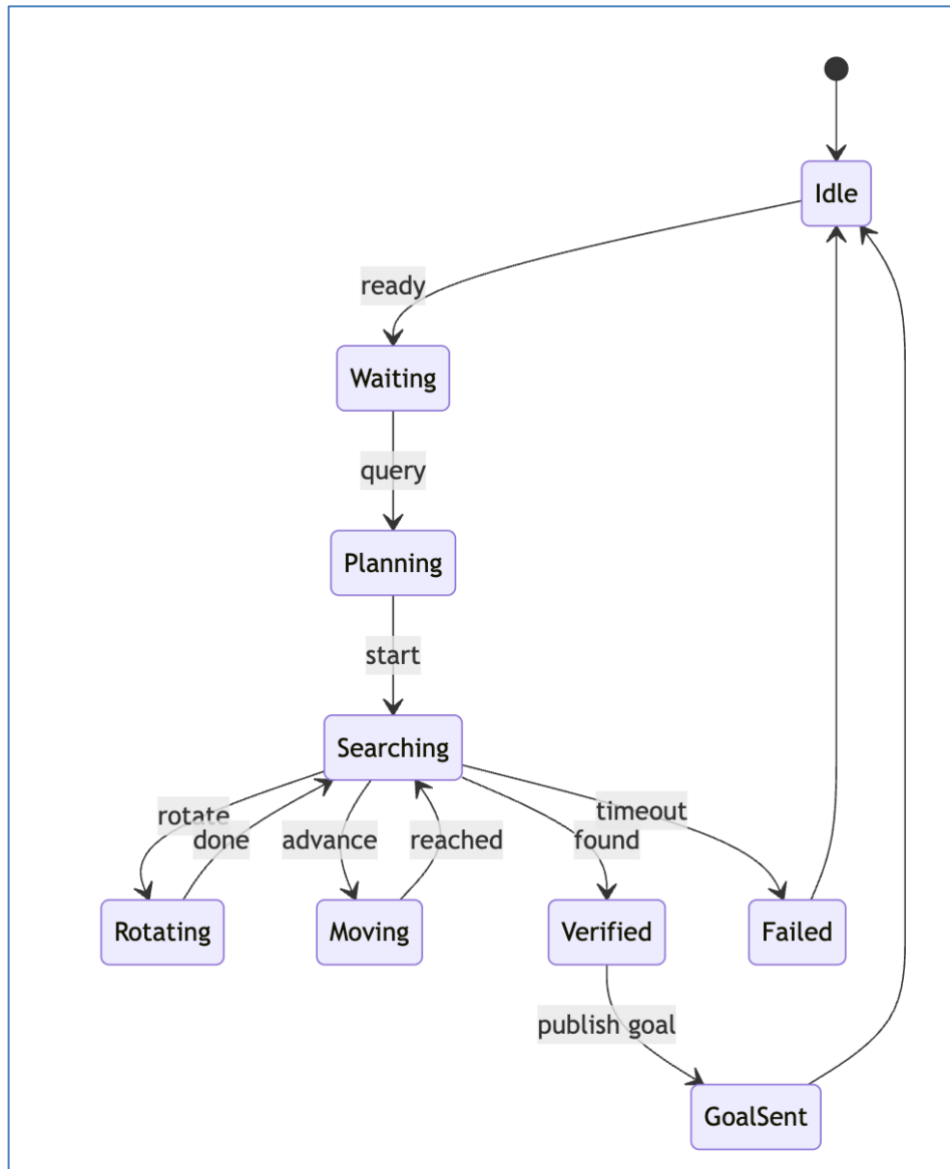


Fig 4.4 Search state-machine diagram



#### Algorithm 4.1: YOLO-only search commander (baseline)

***Input: target class  $c^*$***

***Parameters: rotation step  $\Delta\theta = 45^\circ$ , forward step  $d = 1.0$  m,***

***scan duration  $T_s = 5$  s, standoff distance  $d_{\text{goal}} = 0.5$  m.***

***loop at control rate:***

*receive current RGB frame  $I_t$  and depth image  $Z_t$*

*run YOLOv8 on  $I_t$  to obtain detections  $\{(b_k, c_k, p_k)\}$*

***if any detection with  $c_k = c^*$  and  $p_k \geq p_{\text{min}}$ :***

*choose best detection  $b^*$*

*estimate 3D point  $p_{\text{obj}}$  from  $b^*$  and  $Z_t$*

*set goal position at distance  $d_{\text{goal}}$  in front of  $p_{\text{obj}}$*

*send goal pose to Nav2 via `/goal_pose`*

*wait for Nav2 to finish or timeout*

***else:***

***if current scan time  $< T_s$ :***

*rotate in place by  $\Delta\theta$  spread over  $T_s$  seconds*

***else:***

*move forward by distance  $d$*

*reset scan timer*

The baseline search parameters were selected as pragmatic compromises between coverage, runtime and safety on the physical robot. A  $45^\circ$  rotation step ensures that eight rotations cover a full  $360^\circ$  scan, while still providing sufficient overlap between camera views to avoid missing objects that lie near the boundary between consecutive sectors. The forward distance of 1.0 m was chosen to be large enough to reveal substantially new parts of the environment but small enough to avoid overshooting within typical indoor corridors and laboratory rooms.

During each scan, the robot holds its position for 5 s while rotating slowly, which is long enough to acquire several camera frames and run YOLOv8 multiple times at the available frame rate on the Jetson Orin Nano. The standoff distance of 0.5 m in front of the detected object provides a safety margin that keeps the robot out of collision with the target and nearby obstacles while still placing the object comfortably within view for verification or human interaction. These values were not exhaustively optimised; instead, they were fixed across all experiments to provide a consistent baseline for comparison with the OAK-based and VLM-assisted systems.

The search strategy is intentionally simple and does not model information gain, frontier exploration or probabilistic beliefs about the target's location. More sophisticated exploration strategies could reduce path length or time-to-goal, but they would also introduce additional design variables that are orthogonal to the thesis focus on semantic verification. In this sense, the YOLO-only baseline serves as a conservative reference that reflects how a closed-vocabulary detector is typically integrated with Nav2 on a real robot.

#### 4.3.4 Baseline performance summary

To quantify the performance of the YOLO-only pipeline, experiments were conducted in indoor scenes containing 3 distinct target object instances such as bottles, chairs and boxes. Over a total of 30 runs, the baseline achieved a navigation success rate of approximately 73.33%, where success is defined as reaching a goal pose within a radius 2.5m of the correct object before a global timeout. The false-positive rate, defined as runs in which the robot reached a pose near an incorrect but visually similar object, was 10%, with the remaining runs ending in timeout or planner failure.

These results, although based on a limited number of trials, highlight typical failure modes of the closed-vocabulary baseline: missed detections due to occlusion or viewpoint, and mis-detections where YOLO locks on to an object of the correct class but not the one intended by the user. These weaknesses motivate the VLM-assisted architecture described in Section 4.5, which aims to reduce such errors by incorporating semantic verification of detector proposals using natural-language queries.

### 4.4 OAK Spatial Commander (DepthAI Integration)

Apart from the baseline utilizing the YOLOv8 model, the system also uses a "spatial commander," which leverages the built-in depth and detection functions of the OAK D Lite device to directly acquire 3D coordinates of the objects detected. In this case, the DepthAI pipeline consists of an RGB camera node to feed the color stream, the stereo pair to compute the disparity, and a *StereoDepth* node providing aligned depth data. Specifically, the stereo pair is configured to compute the highest-accuracy depth estimates with sub-pixel disparity and a collection of filters used to minimize noise and flickering of the depth map (spatial, temporal, speckle, and median filters). To build upon this depth estimation pipeline, the *SpatialDetectionNetwork* module is employed, utilizing the yolov6 nano model running on the device with metric coordinates returned along with the detections.

The spatial detector's thresholding takes explicit account of depth limits so that only reliable detections are kept. Specifically, the node uses *setDepthLowerThreshold* to establish a minimum limit so that detections that are too close, often affected by artifacts in sensors and reflected off the robot's body, can be filtered out, and *setDepthUpperThreshold* to set a maximum limit so that detections that are too far away, where the disparity is uncertain and unreliable, can be removed. This has two direct consequences: the suppression of self-detections or detections in proximity to the robot, as well as the use of an optimal distance at which reliable depth information is available.

Thresholds used here correspond to those used for obstacles ranges elsewhere in the system (e.g., point cloud to laser scan mapping).

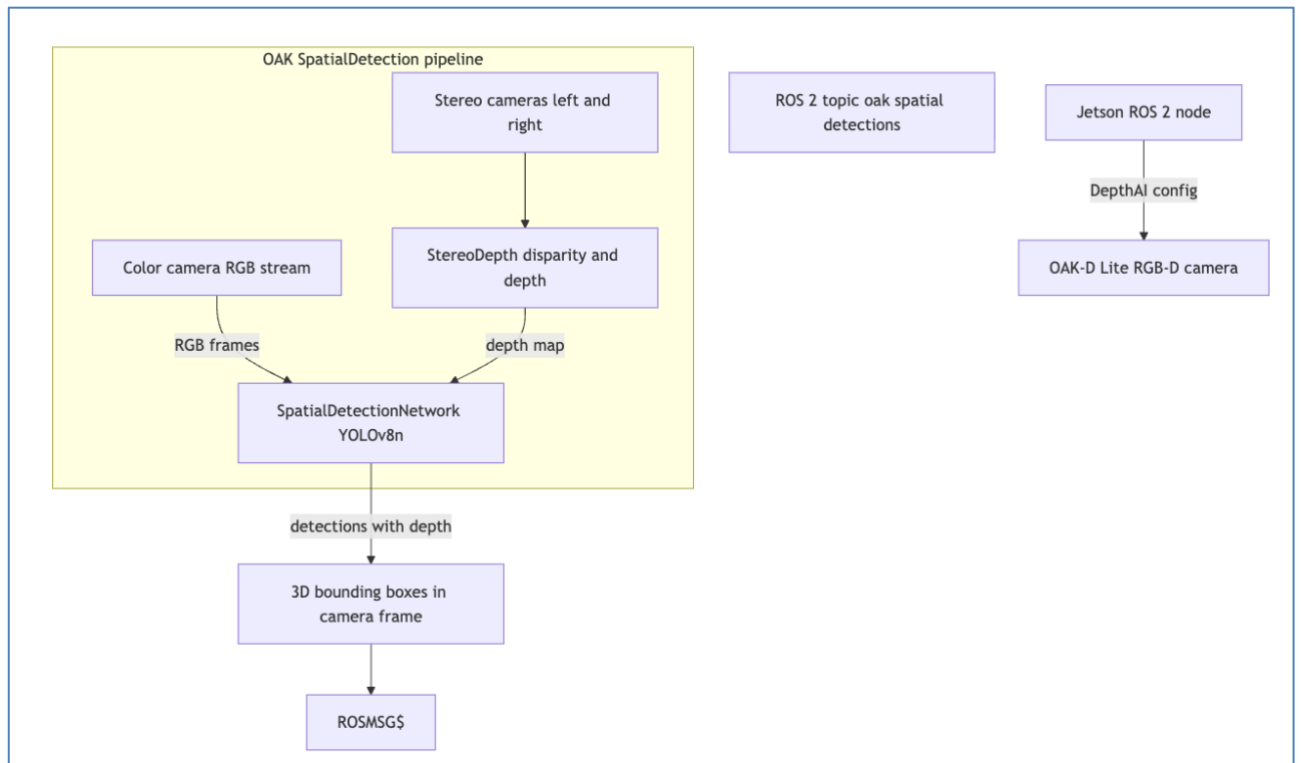


Fig 4.5 OAK SpatialDetection pipeline block diagram

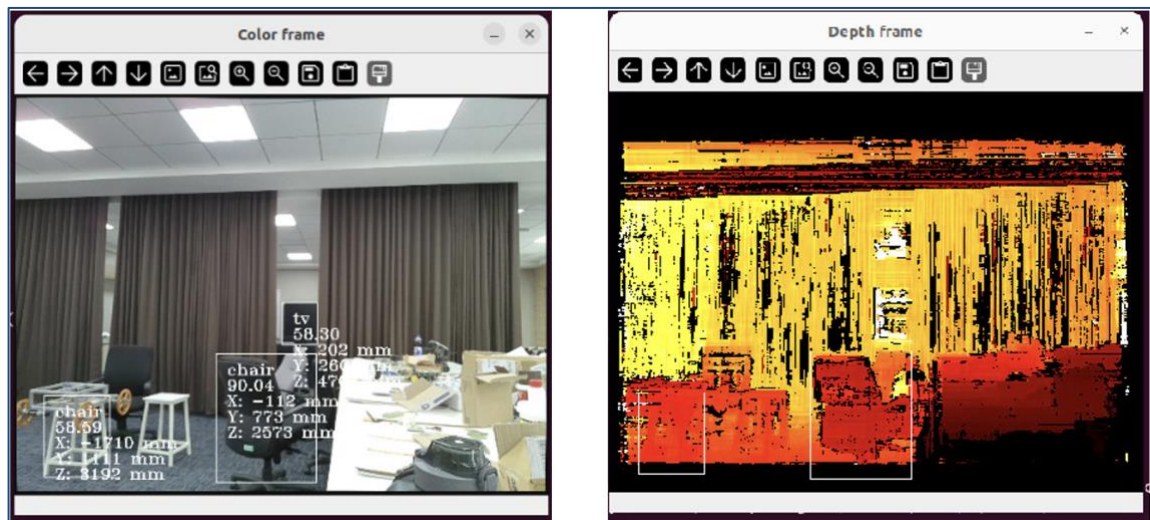


Fig 4.6: “Example scene from the OAK-D camera showing spatial detections (chairs and TV) with metric XYZ coordinates (left) and the corresponding depth map used by DepthAI to compute object ranges (right).”

When the spatial commander receives detections from the OAK pipeline, it employs the *detection.spatialCoordinates* data to find the 3D coordinates of the object in millimeters, using the camera as the origin. According to DepthAI conventions, the z-coordinate is equivalent to forward distance, and the x-coordinate is equivalent to horizontal distance, meaning that the node converts these measurements into meters and translates them to the local frame of the robot based on  $raw\_x = z/1000$  and  $raw\_y = -(x/1000)$ . From this point, it calculates the Euclidean distance between the object

and adds a standoff distance of 0.5 m to ensure that the robot will not collide with the object, but only approach it. When the object is more than 0.5 meters away, the node scales the raw vector and ensures that the result would be a point 0.5 meters away from the object along its line of sight. When the object is closer than the specified distance, a zero-step goal is set (fig 4.5 & 4.6).

The scaled target position is translated into a Nav2 goal in the *base\_link* frame. A *PoseStamped* message is created with *pose.position.x* and *pose.position.y* set to the computed forward and lateral offsets, and the orientation is chosen so that the robot faces the object by computing a yaw angle using  $\text{atan2}(\text{target\_y}, \text{target\_x})$  and converting it into a quaternion. This goal is then published to */goal\_pose*, allowing Nav2 to plan and execute a short, safe motion to a pose where the object is centred in the robot’s view at the desired standoff distance. Because the coordinates come directly from the OAK’s spatial detection network, no explicit projection from 2D image pixels into depth is required in this pipeline.

Compared with the YOLOv8 baseline described earlier, the OAK spatial commander represents a more classical, resource-efficient spatial perception baseline. In the YOLOv8 setup, detection runs on the onboard computer using Ultralytics YOLO, and 3D positions are computed by manually projecting 2D bounding boxes into the stereo depth image using approximate camera intrinsics and depth sampling. In contrast, the OAK-based commander offloads both detection and depth fusion to the OAK hardware, using *yolov6-nano* inside the camera and consuming ready-made 3D coordinates from the DepthAI runtime. This reduces compute load on the Jetson or laptop, simplifies the perception code and provides a clean, self-contained source of metric object locations for Nav2. As such, it serves as the “classical” spatial baseline against which the additional complexity of the VLM-assisted architecture can be evaluated.

## 4.5 VLM-Assisted YOLO Commander

The key contribution of the present work is the *YoloVlmCommander* node implemented in *vlm\_commander.py*, which extends the classical perception and navigation stack with a lightweight vision–language model used as a semantic verification layer. The purpose of this node is not to replace the conventional detector or the Nav2 stack, but to augment them so that the robot can interpret more descriptive user queries and reduce incorrect target selection when the detector alone is not sufficient. In this design, the robot continues to depend on geometric planning and hardware-accelerated object proposals, while the VLM is invoked only to determine whether a candidate region in the current image matches the user’s natural-language target description.

### 4.5.1 Role of the VLM

In the proposed architecture, object proposals are still generated by a conventional detector running on the OAK-D pipeline rather than by the VLM itself. The DepthAI pipeline creates an RGB stream, a stereo depth stream and a *SpatialDetectionNetwork* based on *yolov6-nano*, which produces candidate detections together with metric 3D coordinates in the camera frame. As a result, the commander receives not only the bounding box of each candidate object but also the corresponding *spatialCoordinates*, which are later used to generate a local navigation goal if the candidate is accepted. The novelty of this design lies in using a small VLM as an ROI-based semantic verifier with a tunable confidence threshold and cooldown on top of OAK spatial detections and Nav2, rather than as an end-to-end controller.

The vision–language model, in contrast, performs a narrower semantic task. It does not look at the full camera frame continuously, it does not predict motion commands and it does not generate control actions directly. Instead, whenever OAK SpatialDetection network proposes a potentially interesting object, the commander extracts a cropped region of interest (ROI) around that detection and sends it to the VLM together with the user’s text description of the desired object. The VLM is therefore used as a verifier that answers the question: “Does this candidate crop correspond to the object described by the user?”

This design directly implements the semantic component introduced in the problem formulation, namely the mapping  $S_t = V(I_t, q)$ , where  $I_t$  is the current visual observation and  $q$  is the natural-language query. In practice, the VLM algorithm will be run on a crop of the image as opposed to the full image; hence, the resulting output will be a semantic analysis of a particular object proposal rather than scene understanding. The semantic variable  $S_t$  in this scenario takes the form of a JSON message that includes four fields – *match*, *score*, *reason* and *object\_type* where the *match* field is a Boolean that evaluates to true when the ROI has the targeted object, the *score* is a floating number that lies between 0 and 1, *reason* is text data that justifies why the result was what it was, while *object\_type* is the noun phrase output by the model.

The candidate detections are treated in a very conservative manner here; the first check involves checking that the model returned *match = true* and that the score exceeds the predetermined value *self.vlm\_threshold = 0.70*. Among all the visible detections, only the one with the highest score above the threshold is taken while the others are rejected. Once semantic verification is complete, a navigation goal pose is generated out of the 3D detection. In this manner, the VLM operates simply as a semantics assistant added on top of traditional detection and navigation systems, without being a control mechanism, planner, or reasoning engine.

This separation is central to the overall thesis argument. The proposed system does not attempt to solve robot navigation end-to-end with a large multimodal model, because such a strategy would be computationally expensive, difficult to control and poorly suited to a resource-constrained mobile platform. Instead, the conventional modules remain responsible for the parts they already handle well: DepthAI and YOLO provide fast object proposals and spatial localization, while Nav2 executes safe geometric motion. The VLM contributes only the missing semantic flexibility by validating whether the detected object is actually consistent with the user’s description, including cues such as object category, visible appearance and contextual hints encoded in language.

This role also addresses one of the main limitations of the YOLO-only baseline. In the baseline system, a natural-language query is reduced to a fixed COCO class and any descriptive attributes beyond that class are effectively ignored at decision time. In the VLM-assisted commander, by contrast, the full text description remains available to the semantic module throughout the search process, because the prompt explicitly includes the target description each time an ROI is evaluated. It thus makes the architecture more appropriate to handle an open vocabulary or loosely defined object search problem despite the proposal being based on a closed set detector (fig 4.7).

The other significant outcome of such a structure is that of safety and reliability during operation. The fact that the VLM only operates when a specific verification event occurs means that delays or even failure of the semantic reasoning process cannot jeopardize the stability of the robot’s control cycle. In case of such an outcome, the *YoloVlmCommander* could temporarily halt the robot, request the answer from the model, and choose to either accept the top suggestion or continue verifying its surroundings (fig 4.8) .

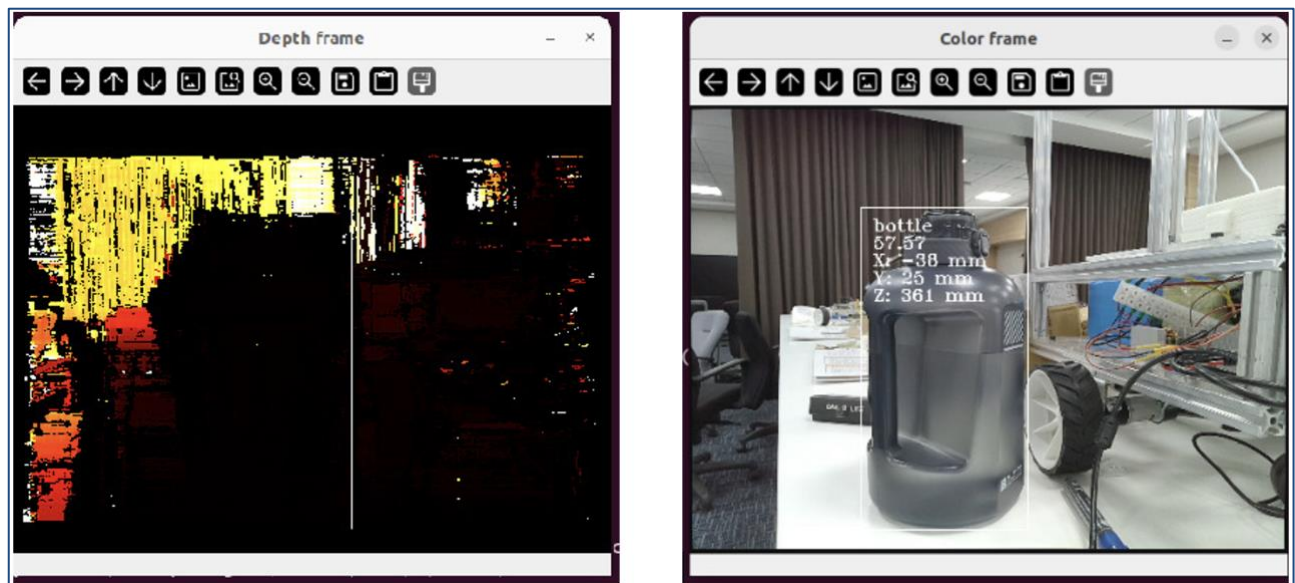
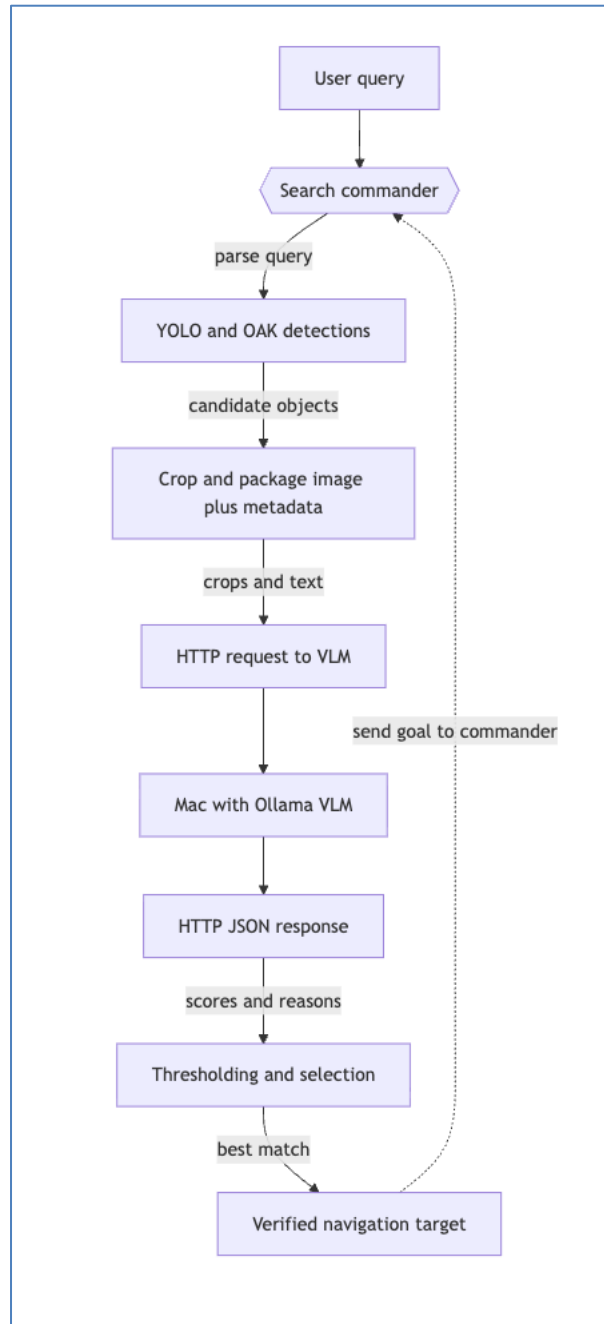


Fig 4.7: “Close-range bottle detection with 3D coordinates from OAK-D (left) and its depth map (right), used as a typical target for VLM-assisted queries.”



**Fig 4.8 VLM Semantic Verification Logic**

#### 4.5.2 OpenAI-Compatible VLM API Integration

In order to incorporate the vision-language model onto the robot without the need for setting up an inference server, the semantic layer of the system takes advantage of the OpenAI-compatible HTTP API offered by Ollama. The YoloVlmCommander node initializes the OpenAI API client using a base URL of the format `http://<MAC_IP>:11434/v1`, with `<MAC_IP>` representing the IP address of the Mac that runs Ollama and exposes the `qwen2.5vl:3b` model on port 11434. This configuration enables the robot to perform multimodal chat completion via the same API as the OpenAI API, except that all the inferences happen locally on the Mac (fig 4.8).

The overall configuration of the system is also straightforward – two machines are used: the Jetson Orin Nano handles ROS 2, DepthAI and the search algorithm, while the Mac runs the VLM

component behind the OpenAI API. In case a good object proposal is received by the robot through the OAK SpatialDetection module, the commander sends the respective image crop along with the description of the desired object by the user in form of a multimodal query to the VLM. The results are delivered in the format of a JSON message, with the values of the match, score, reason and object\_type attributes being treated as a semantic observation for fusion with the YOLO and DepthAI outputs.

#### Algorithm 4.2: VLM-assisted semantic verification

##### **Input:**

- Spatial detections  $D_t$  from OAK (each with bbox  $b_k$ , class id  $c_k$ , spatial coordinates  $(X_k, Y_k, Z_k)$ )
- User query  $q$

##### **Parameters:**

- Minimum crop size  $p_{min} = 50$  pixels
- VLM score threshold  $\tau = 0.70$
- Cooldown interval  $\Delta t_{vlm}$  ( $\approx 2$  s in experiments)
- Standoff distance  $d_{goal} = 0.5$  m

**For each incoming RGB frame and detection set  $D_t$ :**

1. Map each class id  $c_k$  to its COCO label name.
2. Rescale each detection bounding box  $b_k$  to the current image size.
3. Size filtering:

Discard detections whose bbox width  $< p_{min}$  or height  $< p_{min}$ .

**If no detections remain:**

Continue scanning; do not call the VLM.

4. Cooldown check:

**If  $current\_time - last\_vlm\_call < \Delta t_{vlm}$ :**

Skip VLM calls for this frame and return (no semantic decision).

5. Semantic scoring:

Set  $best\_score \leftarrow 0$ ,  $best\_det \leftarrow None$ .



***For each remaining detection  $k$  in  $D_t$ :***

***5.1 Construct a padded crop  $r_k$  around  $b_k$***

*(add 20 pixels margin on each side, clipped to image bounds).*

***5.2 Send  $(r_k, q)$  to the VLM via the multimodal API and***

***receive JSON  $\{match, score, reason, object\_type\}$ .***

***5.3 If  $match == true$  and  $score > best\_score$ :***

*$best\_score \leftarrow score$*

*$best\_det \leftarrow detection\ k$*

*$best\_meta \leftarrow (reason, object\_type)$*

***6. Thresholding and goal generation:***

***If  $best\_det$  is not None and  $best\_score \geq \tau$ :***

*6.1 Read spatial coordinates  $(X, Y, Z)$  of  $best\_det$  from DepthAI.*

*6.2 Convert to robot frame to obtain  $raw\_x$  and  $raw\_y$ .*

*6.3 Compute a goal point at distance  $d\_goal$  in front of the object*

*along the line of sight (or zero-step goal if the object*

*is already closer than  $d\_goal$ ).*

*6.4 Publish the goal as a PoseStamped in  $base\_link$  frame*

*on  $/goal\_pose$  and set  $target\_found = True$ .*

***Else:***

*No detection is accepted in this frame; continue with the*

*exploratory search state machine.*

Since communication with the VLM goes via the Wi-Fi interface and uses dynamic downloading of the model, the semantic call will take some time to process compared to onboard geometric perception, which normally requires an additional few hundred milliseconds of delay. This is the reason why the commander calls the VLM only from time to time using the cooldown mechanism, as defined in Algorithm 4.2. The details of the client implementation and JPEG compression of crops as well as JSON parsing functions can be found in Appendix A.

### 4.5.3 Semantic Verification Logic

The fundamental algorithm of semantic reasoning of the suggested model lies in the `evaluate_detections()` function of the file `vlm_commander.py` (see Algorithm 4.2). The function takes in the RGB stream of the OAK camera along with spatial detections by the `yolov6-nano` model on the device side and further selects those objects according to the goal set by the user; it starts with assigning COCO labels according to class indexes, scaling down detection bounding boxes according to the dimensions of the current frame, and making image patches.

Prior to calling the VLM, the function implements a basic filtering step designed to filter out false-positive detections due to low resolution. The detections with width or height smaller than 50 pixels are removed at this stage. The rationale behind this is that small crops may contain too few features that can be recognized reliably by the VLM. Therefore, it is safer to ignore such regions of interest since most of them do not represent actual objects.

When detections that pass the size filter are received, the function indicates that there is an “interesting object” currently in frame and proceeds to evaluate the cooldown period. If the current timestamp falls within the cooldown interval represented by `vlm_cooldown`, the function will not proceed with semantic classification to prevent rapid querying of the VLM with new detections. When the cooldown period has elapsed, the image is cropped based on the bounding box coordinates and adding a padding margin of 20 pixels to all four directions. The resulting cropped image, called `cropped_roi`, will be a little larger than the actual ROI to provide more context information for the VLM in case part of the object is occluded or there is additional contextual information in its vicinity. An empty crop leads to ignoring the candidate.

Once the ROI is obtained successfully, the commander temporarily stops the robot by publishing a null *Twist* value and requests the VLM to analyze the detected object based on the cropped image and target description. This step is recorded as follows “YOLO proposed [object\_name]. Asking VLM to match description...” in the ROS logs and it is useful to conduct analysis since it allows knowing which object proposals were semantically evaluated. In turn, the returned information in JSON format is converted to the following fields: *match*, *score*, *reason* and *object\_type* and stored in the following attribute – *self.last\_vlm\_result*. Another ROS log message is generated and displays the result of the semantic evaluation process in the following concise format: [match], [score], ([reason]) “[VLM’s Noun Phrase]”. These logs help in conducting further analysis in case something goes wrong, since it allows distinguishing between the failures made either by the detection model YOLO or the semantic model VLM.

During this loop, the function retains values *best\_score*, *best\_detection*, *best\_reason* and *best\_name* while considering all detections found in the frame. In case the currently considered ROI was tagged as match with a confidence score greater than the current best score, it is chosen as a better choice. After the loop completes, the function returns a semantic best candidate with its score and meta data back to the *vision\_loop()*. This implies that, regardless of how many objects are found within the image, only one, the one with the highest score, will be used for navigation.

Finally, the VLM confirmation check is performed inside *vision\_loop()* where the value of *best\_score* is tested against *self.vlm\_threshold = 0.70*. If the value of *best\_detection* is other than None and *best\_score* is greater than or equal to 0.70, the VLM confirms the target found and logs such confirmation before executing *navigate\_to\_detection(best\_detection)*. Within that function, the coordinates of the target in space are used to calculate a local goal point (standoff of 0.5 m) which is sent to Nav2 and *target\_found* is set to *True*. Therefore, a target does not get accepted simply because YOLO saw an object of certain class; the semantic VLM must validate it

The threshold value of 0.70 was selected as a reasonable balance between false positive identification and non-identification. Setting the threshold value too low would allow the robot to accept ROIs that visually appear to be good matches but actually direct it toward the wrong target. This can be an issue in cases where there are multiple items in view and some are either very similar visually or share semantic features. On the other hand, setting the threshold value too high could cause the system to be excessively cautious about accepting ROIs that are only partially covered or poorly lighted. The 0.70 threshold level was determined during repeated experimentation inside of the laboratory. This selection represents the system's overall design philosophy. The role of the VLM is not considered entirely accurate, and therefore the information is filtered by heuristic means before a navigation command is issued.

#### 4.5.4 Exploratory Search State Machine with VLM Assistance

It should be noted that the *YoloVlmCommander* node does not take into account the case where the target object is right there in the field of vision of the camera from the very beginning. The node uses a specific search state machine, which includes both local exploration, closed-loop turns, and limited relocation moves in order to analyze the entire surroundings systematically. Such an approach to searching for objects is borrowed from the previous YOLO only baseline. The reason for retaining the baseline search policy in the new hybrid approach is to avoid introducing the new search policy while adding semantic verification.

In terms of the highest level state machine, the whole system comprises two clearly stated states – *STATIONARY\_SCAN* and *WAIT\_FOR\_MOVE*, along with an implicit state called turning that is regulated by the Boolean variable *self.turning*. The initial state is *STATIONARY\_SCAN*. The robot stays still and collects data about the environment for the current sector through the use of OAK RGB camera frame and DepthAI spatial detections, as well as VLM semantic verification when required. The amount of time spent on this stage equals *self.scan\_duration = 10.0*, which means that the robot spends 10 seconds scanning one sector until it realizes that there is nothing interesting there and switches to scanning another one. Note that the dwell time is larger compared to the one used in case of YOLO only approach due to the fact that VLM queries take some more time.

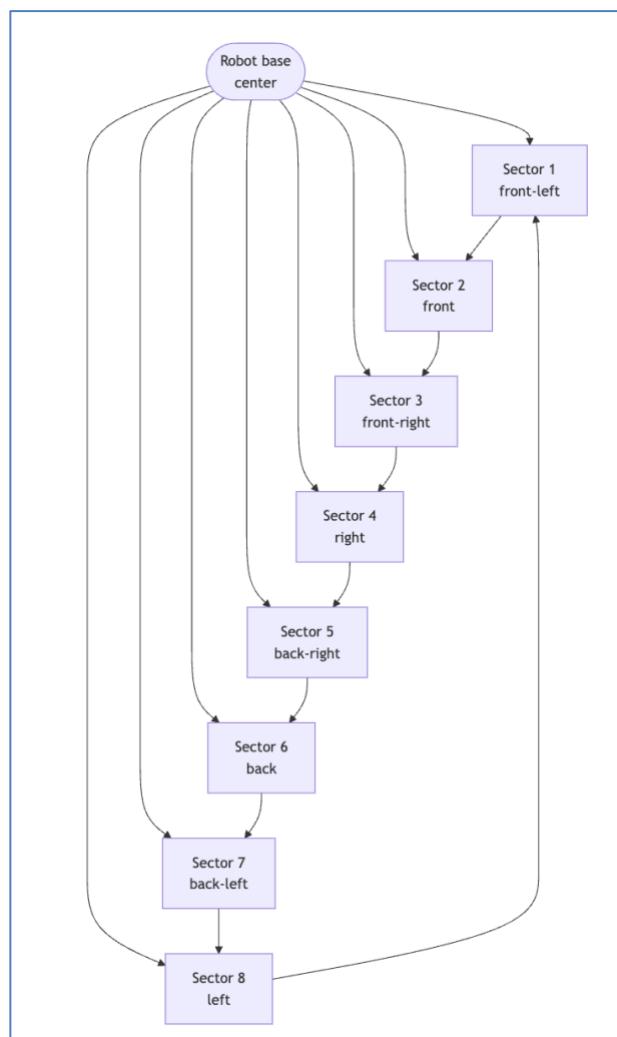
Another clearly defined state is called *WAIT\_FOR\_MOVE* and it is set when the robot has performed the entire scan of 360 degrees and has ordered itself to move forward into a new location. The purpose of this state is that the robot does not perform any scanning or moving action. All it has to do is wait until the timestamp contained in *self.goal\_wait\_until* has passed. This allows the robot to finish up its previous order and proceed with the perception-based search. Upon completing this wait state, the node announces that the robot has reached the new vantage area and proceeds with transitioning to the *STATIONARY\_SCAN* state once more.

Mode three is turning mode, which is not defined as one of the string states but is controlled using the flag *self.turning*. If this flag is *True*, the robot stops all the scanning process and the relocation algorithm to rotate in a closed circle through a 45° angle using feedback from the */odom* topic. This operation is done using the method *start\_turn\_45()* that will increase the total number of turns done by the robot, calculate the new desired angle by adding *self.turn\_angle\_deg = 45.0* to the angle currently reached and then wrapping the angle to  $[-\pi, \pi]$  using  $\text{atan2}(\sin(), \cos())$ .

Each time *vision\_loop()* executes a cycle, provided *self.turning* is set, the commander calculates the yaw error between *self.target\_yaw* and the current yaw based on the odometry reading, once again wrapping the error into the principal value of the angle. The control law then becomes a proportional one in which the robot's angular speed instruction is obtained by multiplying the size of the yaw error while bounding the latter to be within  $[0.15, \text{self.angular\_speed}]$  rad/s where *self.angular\_speed* =

$0.5 \text{ rad/s}$ . In this manner, the angular speed decreases as the robot gets closer to its final orientation. The turning process stops when the absolute yaw error goes under  $0.05 \text{ rad}$ , which is equivalent to  $\sim 3^\circ$ . The robot then stops, *self.turning* is reset, and a brief delay is imposed using *self.paused\_until* while the node switches back to *STATIONARY\_SCAN* with an updated *scan\_start\_time* in order to check the new sector.

The algorithm of sector search depends on *self.turn\_count* and *self.max\_turns\_before\_move = 8*, representing 8 sectors of 45 degrees around the robot. In *STATIONARY\_SCAN*, if the robot fails to detect a valid target, while the scan duration exceeds *self.scan\_duration*, then the node reports a clean sector and increases the number of turns made so far. If this number remains less than eight, the commander simply makes another turn and scans the next sector. The algorithm can be understood as a search technique where the robot takes samples from the environment based on a particular angular position around itself (fig 4.9).



**Fig 4.9 polar sectors sketch**

In the event that all eight sectors have been analyzed without finding a suitable target, then the node calls upon the *move\_to\_new\_vantage\_point()* method. The *move\_to\_new\_vantage\_point()* function is responsible for updating the relocation count, setting a *PoseStamped* goal for a distance of 1.0 m ahead in the *base\_link* frame while setting *self.goal\_wait\_until = time.time() + 8.0*. The value of *self.turn\_count* is also set to zero at this point as well as setting the state to *WAIT\_FOR\_MOVE*,

thereby allowing the robot to initiate a new sector sweep starting from a new location once it completes its forward movement.

The significance of this state machine comes from the fact that it implements the physical exploration process on top of which the semantic reasoning can be applied. The VLM does not make decisions about the direction and location in which the robot should turn; rather, it is the role of the state machine to ensure that the robot gets a series of meaningful views, and the task of determining whether any of the identified objects from these views corresponds to the user's description is performed by the VLM.

#### 4.5.5 Detection Time Measurement

In order to measure the responsive capability of the VLM aided robot, the command record a simple detection time value for every experiment. The purpose of such a value is to measure how much time the robot takes since the start of the experiment up until the point when it finds its first semantically valid object which leads to navigation towards that particular object. Since we can use the exact same concept for the YOLO-only approach as well, it serves as a basis for comparison.

The measurement starts when the instance of *YoloVlmCommander* gets created. At initialization, the member variables *self.trial\_start\_time* is assigned by *time.monotonic()* and *self.detection\_time\_sec* is set to *None*. *Time.monotonic()* is an appropriate function call in this context, because it supplies a monotonically-increasing source of timing information that is immune to changes in system clock times, which makes it ideal for measuring elapsed time between certain events during robotic experiments. Intuitively speaking, this is the time stamp of the search trial beginning, or in other words, when the robot starts its VLM assisted search routine.

The real value is registered in the *navigate\_to\_detection()* method, which gets invoked only when the object candidate has been confirmed to be semantically correct and the program is prepared to convert the detection into the Nav2 goal. The very beginning of the function contains a condition that verifies whether *self.detection\_time\_sec* remains as *None*. If yes, the difference between current *time.time.monotonic()* and *self.trial\_start\_time* gets registered as *self.detection\_time\_sec*, as well as printed out to ROS in the following format: *DETECTION TIME: ... s*. Thus, the value gets registered only once, which means that in case of further invocation of the same method during one trial session, the measurement won't be overwritten.

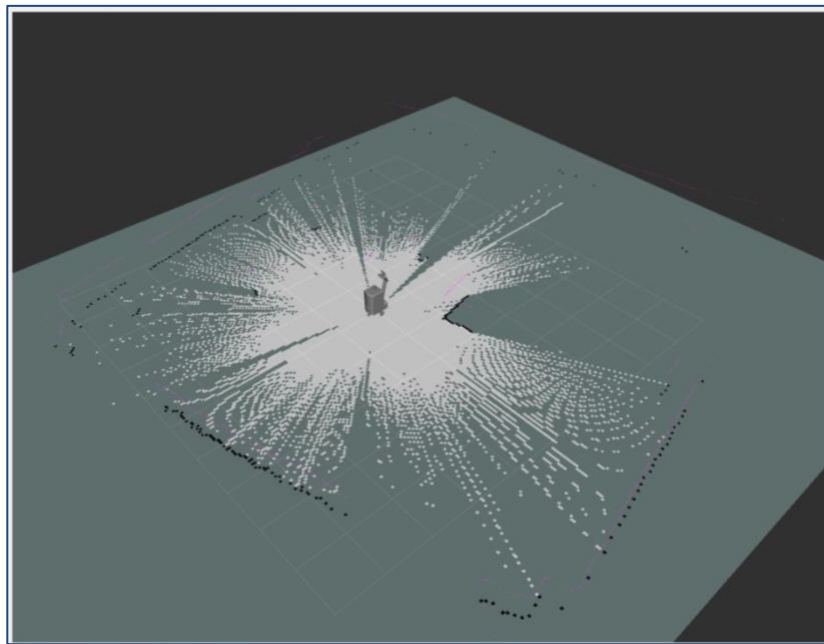
The benefit of such an approach lies in that this metric will correspond to the real life of the hybrid system. The recorded time accounts not only for the time required for the processing of detector data, but also for the sector sweep time, the 45° turn time, the wait time for VLM queries, the optional wait time after *vlm\_cooldown*, and, eventually, getting a reliable semantic signal to confirm the decision to navigate towards the detected target. That is why this metric should be considered at the system level as a target acquisition time.

In the evaluation section coming later, the above value of *detection\_time\_sec* may serve as the principal measure of detection time for the VLM assisted pipeline and can be used for direct comparison with another similar measure obtained by running the YOLO only baseline test. This comparison would be necessary since although the addition of VLM might have enhanced semantic accuracy in the task, there could have been additional delay introduced owing to inference over Wi Fi, as well as validation of each of the detected regions. Therefore, using detection time as a metric, we will be able to gauge this trade-off between increased semantic reasoning and target acquisition delay.

## 4.6 Mapless Nav2 Stack

The suggested architecture leverages the typical ROS 2 Navigation2 stack in its mapless version to run all the motion-related commands provided by the perception and semantics levels. Under this approach, the launch file of the Nav2 package (called *mapless\_nav2.launch.py*) launches the Nav2 components and links them to the robot bringup with a common parameter file, whereas the commander components work with Nav2 solely through the goal topic and the velocity interface. As a result, the thesis can concentrate on developing perception and semantics-based guidance mechanisms, making use of an established navigation framework for planning and behavior generation.

This description defines the initialization of the core Nav2 components as nodes, namely *controller\_server*, *planner\_server*, *behavior\_server*, *bt\_navigator*, *smoother\_server*, *velocity\_smoother*, and *lifecycle\_manager\_navigation*. While the *controller\_server* handles local trajectory following and closed-loop velocity control, the *planner\_server* plans the global collision-free path from the robot's starting pose to its target pose. The recovery and other high-level behaviors such as the costmap cleaning or backward movement are handled by the *behavior\_server*, whereas the navigation decision-making is executed by the *bt\_navigator* component which makes decisions on when to plan, follow a path, and perform recoveries through the use of a behavior tree. In addition, the *smoother\_server* generates dynamic trajectories from the planned paths and the *velocity\_smoother* processes the velocities before the execution in the base. Lastly, the *lifecycle\_manager\_navigation* manages the whole process by configuring and activating the mentioned components.



**Fig 4.10 Mapless Nav2 Stack**

All the navigation nodes have a common set of parameters that is passed from *nav2\_params.yaml* found under the path *Urdf\_model\_description/config/nav2\_params.yaml*. These parameters include the costmap, plugins for the controller and planner, obstacle layers, robot footprint among others which should be uniform throughout the stack. Passing the same configuration file to each of the nodes in the Nav2 stack guarantees that global and local navigation will use the same robot

configuration. This will make it easy to experiment since any change in the parameters affects the whole stack (refer fig. 4.10).

The integration with the rest of the robot is handled through deliberate topic remappings of the velocity command interface. Within the Nav2 stack, both the *controller\_server* and the *behavior\_server* publish velocity commands on an internal topic remapped to *cmd\_vel\_nav*. The *velocity\_smoother* node subscribes to this internal navigation velocity topic and outputs a smoothed command on the standard */cmd\_vel* topic that is actually consumed by the base. In the launch file, this is achieved by remapping *cmd\_vel* to *cmd\_vel\_nav* on the Nav2 controllers and by remapping *cmd\_vel\_smoothed* back to *cmd\_vel* in the velocity smoother. Hence, both the commander nodes at the upper hierarchy, including the YOLO baseline and the VLM aided commander, have the ability to send their respective emergency stop messages and corrections directly to the */cmd\_vel* topic if necessary, whereas Nav2 continues to have its own namespace for velocities and smoothness component. The design makes sure that all the perception and semantic processing components integrate smoothly with a typical Nav2 workflow.

## 4.7 Bringup and Pipelines

The complete experimental system is assembled through a layered bringup process in which the robot description, base interfaces, sensing stack, odometry and navigation are started first, followed by the specific experiment node for the perception configuration under test. Two bringup launch files are used in practice: *robot\_bringup.launch.py* when the OAK-D camera pipeline is required, and *no\_cam\_bringup.launch.py* when only the base, LiDAR and odometry stack are needed. This split makes it possible to run both full RGB-D experiments and reduced bringup configurations without modifying the rest of the software architecture.

In both bringup variants, the robot model is loaded from the URDF/Xacro description and published to ROS through *robot\_state\_publisher*, while *joint\_state\_publisher* or *joint\_state\_publisher\_gui* is launched depending on whether the GUI option is enabled. A static transform between *base\_footprint* and *base\_link* is also published so that the robot frame tree remains well defined for Nav2, odometry and sensor fusion. The differential-drive base is interfaced through *noodom\_diff\_drive\_bridge*, which exposes the velocity interface used by Nav2 and by the higher-level commander nodes. These components form the common geometric backbone of the system regardless of which perception module is active.

The LiDAR and odometry pipeline is also common to both bringup files. The RPLIDAR node is launched with the frame name *laser\_frame*, and its */scan* output is used by the RF2O laser odometry node, which publishes */odom* and the corresponding transform between *odom* and *base\_footprint* at 10 Hz. This odometry stream is the primary pose source used by the commander nodes for closed-loop turning and by Nav2 for local motion execution. In this sense, *no\_cam\_bringup.launch.py* is a minimal bringup path that activates the robot model, the differential-drive interface, LiDAR sensing and odometry but leaves out the depth camera and point cloud processing.

The full bringup path in *robot\_bringup.launch.py* extends this configuration with the OAK-D Lite perception stack. The camera is launched through *depthai\_ros\_driver* using the *oak\_pcl.yaml* configuration file, with the parent frame set to *oak\_data\_frame* and IMU disabled. On top of the camera container, a composable *depth\_image\_proc::PointCloudXyzNode* converts the aligned stereo depth and rectified intensity image into a point cloud published on */oak/points*. That point cloud is then processed by *pointcloud\_to\_laserscan*, which filters points by height and range and publishes a derived scan on */oak/scan* representing frontal obstacles seen by the camera. A stereo

visual odometry pipeline is present in the file but intentionally disabled, since RF2O laser odometry is used as the active odometry source in the actual experiments.

After the base and sensing bringup is active, the next layer is the mapless navigation stack described in the previous subsection. In practice, *mapless\_nav2.launch.py* is launched after robot bringup so that Nav2 can connect to the already available */odom*, */scan*, TF tree and velocity interface. This results in a modular two-stage startup process: first establish the robot and perception interfaces, then activate the planner, controller, behavior tree navigator and velocity smoother. Once these services are running, the system is ready to accept local goals from whichever experiment node is selected.

The final step is launching the specific experiment node corresponding to the perception strategy being evaluated. For the YOLO-only baseline, the *ros2\_yolo\_vlm.py* node is run, even though in this configuration it functions purely as a YOLO search commander with language parsing and no VLM calls. For the classical spatial baseline, the DepthAI-based *spatial\_node.py* is used to verify on-device spatial detections from *yolov6-nano* running on the OAK hardware. For the main proposed method, the *vlm\_commander.py* node is launched; this node internally builds its own DepthAI pipeline, receives OAK spatial detections, queries the remote Qwen2.5-VL model through Ollama and publishes confirmed target goals to Nav2.

The practical launch sequence used in experiments therefore follows a fixed order. First, the robot is brought up using *robot\_bringup.launch.py* for camera-enabled experiments or *no\_cam\_bringup.launch.py* for LiDAR-only operation. Second, the mapless Nav2 stack is started using *mapless\_nav2.launch.py*, which activates the planner, controller and behavior servers with the shared navigation parameter file. Third, one of the experiment nodes is launched manually depending on the trial type: *ros2\_yolo\_vlm.py* for the YOLO baseline, *spatial\_node.py* for the OAK spatial baseline, or *vlm\_commander.py* for the VLM-assisted system. This separation of bringup, navigation and experiment logic makes the system easy to reproduce and also ensures that the same robot and Nav2 infrastructure is reused across all compared methods.

## 4.8 Implementation Summary and Design Choices

One of the key choices made in the design of this project was to place the vision-language model external to the real-time motion control loop. The robot cannot employ the VLM to set velocities, select angular rates, and navigate its way around obstacles, but motion control stays in the classic ROS 2/Nav2 ecosystem, while the VLM is employed solely to semantically confirm that the candidate object is consistent with the user's verbal specification of what they are looking for. This decision was driven by practical considerations of safety and time sensitivity. Motion control needs to operate with reliable updates, which cannot be guaranteed for the VLM due to the fact that the former is connected through Wi-Fi via HTTP API, and image processing and model inference do add non-deterministic delays into the chain. Allowing the latter to directly influence the motion controller introduces uncertainty and the potential for instabilities during motion control operations like turning and avoiding obstacles.

A second important design choice was to rely on OAK-based spatial detections instead of reconstructing object positions from a separate ROS depth image pipeline on the host computer. In the implemented VLM commander, the OAK device runs a SpatialDetectionNetwork built around *yolov6-nano*, and each detection already includes spatial coordinates together with the image bounding box. This is a practical advantage because the OAK platform's RVC2 hardware is designed for embedded AI and DepthAI pipelines, with on-device neural inference capability and dedicated computer vision processing resources. Using the hardware-accelerated spatial detections reduces the amount of perception computation that must be performed on the onboard host, avoids an extra



projection stage from 2D detections into host-side depth maps, and keeps the target localization path compact and efficient. For a mobile robot with limited onboard compute, this is preferable to a heavier architecture in which RGB detection, depth alignment and 3D projection are all handled centrally in ROS.

Similarly, the decision to use VLM externally on another machine (the Mac) and not onboard the robot itself is a straightforward choice. The robot already is tasked with base control, running ROS 2, managing cameras, processing LiDAR data, odometry, and Nav2. Adding a visual-language model to that same machine would introduce extra compute load and potentially even affect the timings of other robot tasks. As such, a semantic model in the form of qwen2.5vl:3b is hosted externally via Ollama's API compatible with OpenAI services; ROIs are sent sporadically via Wi-Fi for confirmation. Notably, the selected model, as well, is considerably smaller in comparison with other models of similar kind, 7B, 32B and 72B, respectively: according to the information on the Ollama model card, qwen2.5vl:3b has around 3.75 billion parameters, making it far easier to deploy locally or near-edge compared to the aforementioned bigger models. Besides being easier to manage due to smaller memory requirements, smaller start-up cost, lower inference time, using ROIs and not whole pictures also makes the task less bandwidth-heavy per call. Thus, it can be said that the current design is not striving for maximally possible semantic capacity, but for "good enough" semantics on a realistic hardware platform.

This design choice also includes many explicit omissions. For instance, the system does not create a semantically rich map of its surroundings. It neither creates object hypotheses in an ongoing way using a persistent scene graph nor incorporates the linguistic information into a globally coherent semantic topography or memory model. Rather, the system employs a reactive approach by scanning the current sector, verifying detected candidates, turning around, and repeating the process if required while moving on to another position. In addition, the navigation layer does not represent an uncertainty-aware system for planning that works in terms of explicitly defining semantic uncertainty in the workspace and planning viewpoints accordingly using an information-theoretic approach.

Omission of these approaches is deliberate since they reflect the practical stance of the approach vis-a-vis related research. Many sophisticated robotic language-guided approaches involve heavy usage of semantic memory, multimodal models with huge amounts of parameters, complex visual scene understanding approaches, and uncertain exploration planners – all of which may be hard to implement in a robust manner on limited hardware and computing power of small mobile robots. In contrast, this approach seeks to address a smaller but more practical issue: can a simple object-search robot benefit from a simple semantic VLM module added onto a normal ROS 2 navigation stack? The exclusion of dense semantic maps and global exploration planning makes this approach straightforward to understand, reproduce and implement while demonstrating the usefulness of semantic filtering via language in the context of real robotic object search.

## Chapter 5 : Results and discussion

### 5.1 Experimental procedure

The evaluation was conducted in an indoor laboratory environment using the same differential-drive robot, sensing stack, and mapless Nav2 configuration described in Chapter 4. Two search configurations were compared: a YOLO-only baseline implemented in `ros2_yolo_vlm.py`, and the proposed VLM-assisted configuration implemented in `vlm_commander.py`. In both cases, the robot executed the same underlying exploration policy so that differences in performance can be attributed mainly to the perception and semantic layer rather than to changes in motion strategy.

For each object category, ten trials were performed for the YOLO-only commander and ten trials were performed for the VLM-assisted commander, giving 30 trials per method across the three categories: chair, bottle, and person. Each trial began with the robot at a fixed starting pose, after which the user entered a target description. In the baseline condition, this query was parsed into a COCO-compatible target class and any extra descriptive words were stored as attributes but were not used for visual reasoning during target validation. In the VLM-assisted condition, the full natural-language description was preserved and passed to the semantic verification layer whenever a candidate ROI was evaluated.

The search procedure followed a sector-based exploration policy in both methods. The robot first remained stationary and scanned the current sector; if no valid target was accepted, it performed a closed-loop  $45^\circ$  turn using odometry feedback and scanned again. After eight such turns, corresponding to a full  $360^\circ$  sweep, the robot moved forward by 1.0 m to a new vantage point and repeated the scan-turn cycle. In the YOLO-only node, the sector scan duration was 5.0 s, whereas in the VLM-assisted node it was increased to 10.0 s to allow time for semantic verification and network-delayed VLM responses.

A trial was considered successful when the robot accepted a target and navigated to the corresponding local Nav2 goal with a 0.5 m standoff from the object. For each trial, the following raw values were recorded: success or failure, detection time in seconds, reach time in seconds, number of  $45^\circ$  turns, number of 1.0 m moves to new vantage points, and notes when relevant. Detection time was measured from the beginning of the trial to the first accepted target; in the VLM-assisted system, this corresponds specifically to the first semantically verified detection whose score exceeded the threshold of 0.70. Failures were also recorded with notes when identifiable, for example a baseline person trial marked “nav2 failed” and a VLM-assisted person trial marked “False positive.”

The VLM-assisted trials additionally tested the ability of the semantic layer to use descriptive natural language rather than only object category labels. Chair queries included examples such as “a black chair with a bag on it,” “a white chair,” “a grey chair with whitish tone,” and “a black chair that appears closer than grey chair,” which required the system to distinguish between multiple instances of the same object category using appearance-level cues. Because the VLM prompt explicitly instructs the model to consider object category, color, shape, and visible context when judging a crop, the semantic layer could disambiguate between chairs of the same class and guide the robot toward the instance that best matched the description, such as selecting the blue or black chair rather than another chair in the same frame when color-based language was used (refer table 5.1 & 5.2).

Table 5.1 — Raw trial outcomes for the YOLO-only commander

| Label & query | success | detect_time_s | reach_time_s | Turns | Moves | success_flag |
|---------------|---------|---------------|--------------|-------|-------|--------------|
| chair         | yes     | 4.33          | 10.24        | 1     | 0     | TRUE         |
| chair         | yes     | 4.69          | 9.94         | 1     | 0     | TRUE         |
| chair         | yes     | 1.69          | 7.89         | 0     | 0     | TRUE         |
| chair         | yes     | 3.43          | 22.22        | 0     | 0     | TRUE         |
| chair         | no      | 2.15          |              |       |       | FALSE        |
| chair         | yes     | 2.34          | 15.73        | 0     | 0     | TRUE         |
| chair         | no      | 2.97          |              |       |       | FALSE        |
| chair         | no      | 3.41          |              |       |       | FALSE        |
| chair         | yes     | 4.12          | 25.43        | 0     | 0     | TRUE         |
| chair         | no      | 2.57          |              |       |       | FALSE        |
| bottle        | yes     | 10.25         | 15.46        | 2     | 0     | TRUE         |
| bottle        | yes     | 22.65         | 10.29        | 4     | 0     | TRUE         |
| bottle        | yes     | 88.35         | 17.46        | 9     | 1     | TRUE         |
| bottle        | yes     | 112.44        | 14.72        | 20    | 3     | TRUE         |
| bottle        | yes     | 24.76         | 5.26         | 3     | 0     | TRUE         |
| bottle        | yes     | 34.01         | 16.33        | 4     | 0     | TRUE         |
| bottle        | no      |               |              | 21    | 3     | FALSE        |
| bottle        | no      |               |              | 18    | 3     | FALSE        |
| bottle        | yes     | 44.74         | 20.22        | 5     | 0     | TRUE         |
| bottle        | no      |               |              | 25    | 3     | FALSE        |
| person        | yes     | 0.95          | 6.57         | 0     | 0     | TRUE         |
| person        | yes     | 17.44         | 7.34         | 2     | 0     | TRUE         |
| person        | yes     | 0.88          | 5.31         | 0     | 0     | TRUE         |
| person        | yes     | 23.24         | 12.86        | 3     | 0     | TRUE         |
| person        | yes     | 30.68         | 17.33        | 4     | 0     | TRUE         |
| person        | yes     | 8.87          | 13.01        | 1     | 0     | TRUE         |
| person        | no      | 16.7          |              | 2     | 0     | FALSE        |
| person        | yes     | 46.41         | 16.94        | 6     | 0     | TRUE         |
| person        | yes     | 66.75         | 19.05        | 7     | 0     | TRUE         |
| person        | yes     | 105.81        | 5.32         | 12    | 1     | TRUE         |

Table 5.2 — Raw trial outcomes for the VLM-assisted commander

| Label | query                          | success | detect_time_s | reach_time_s | Turns | Moves | success_flag |
|-------|--------------------------------|---------|---------------|--------------|-------|-------|--------------|
| chair | a black chair with a bag on it | yes     | 8.71          | 10.06        | 0     | 0     | TRUE         |
| chair | a bag on a black chair         | yes     | 21.37         | 11.32        | 2     | 0     | TRUE         |
| chair | a black chair                  | yes     | 22.16         | 13.41        | 2     | 0     | TRUE         |

|        |                                                   |     |        |       |    |   |       |
|--------|---------------------------------------------------|-----|--------|-------|----|---|-------|
| chair  | a black chair that appears closer than grey chair | yes | 19.81  | 9.06  | 0  | 0 | TRUE  |
| chair  | a black chair facing its back                     | no  | no     | no    | 8  | 1 | FALSE |
| chair  | a white chair                                     | yes | 10.16  | 8.33  | 0  | 0 | TRUE  |
| chair  | a white chair chair with grey tone                | yes | 8.6    | 10.44 | 1  | 0 | TRUE  |
| chair  | a grey chair with whitish tone                    | yes | 7.68   | 15.21 | 0  | 0 | TRUE  |
| chair  | a white chair without a bag on it                 | yes | 30.89  | 22.32 | 2  | 0 | TRUE  |
| chair  | a chair of white color                            | yes | 16.66  | 40.44 | 1  | 0 | TRUE  |
| bottle | a bottle                                          | yes | 20.91  | 10.33 | 0  | 0 | TRUE  |
| bottle | a green botte                                     | no  | no     | no    | 23 | 2 | FALSE |
| bottle | bottle                                            | yes | 19.77  | 1     | 1  | 0 | TRUE  |
| bottle | a bottle of 1L                                    | no  | no     | no    | 13 | 1 | FALSE |
| bottle | a bottle of 1L                                    | yes | 89.33  | 12.33 | 1  | 0 | TRUE  |
| bottle | a green bottle with slight green cap              | no  | no     | no    | 27 | 2 | FALSE |
| bottle | a green bottle with slight green cap              | yes | 112.78 | 15.64 | 10 | 1 | TRUE  |
| bottle | a botte of light green color                      | no  | no     | no    | 14 | 1 | FALSE |
| bottle | a botte of light green color                      | no  | no     | no    | 17 | 2 | FALSE |
| bottle | a green bottle of one approx litre                | yes | 55.33  | 6.33  | 0  | 0 | TRUE  |
| person | person                                            | yes | 3.62   | 24.73 | 0  | 0 | TRUE  |
| person | a tall person                                     | yes | 5.38   | 20.22 | 0  | 0 | TRUE  |
| person | a person wearing green tshirt                     | yes | 40.2   | 36.12 | 3  | 0 | TRUE  |
| person | a person sitting                                  | yes | 5.39   | 21.6  | 0  | 0 | TRUE  |
| person | a green tshirt wearing person                     | yes | 42.16  | 25.16 | 3  | 0 | TRUE  |
| person | a person with black hair                          | yes | 108.58 | 35.42 | 8  | 0 | TRUE  |
| person | a person facing forward                           | yes | 21.37  | 19.79 | 1  | 0 | TRUE  |
| person | a person sitting with a laptop                    | yes | 60.55  | 26.22 | 4  | 0 | TRUE  |

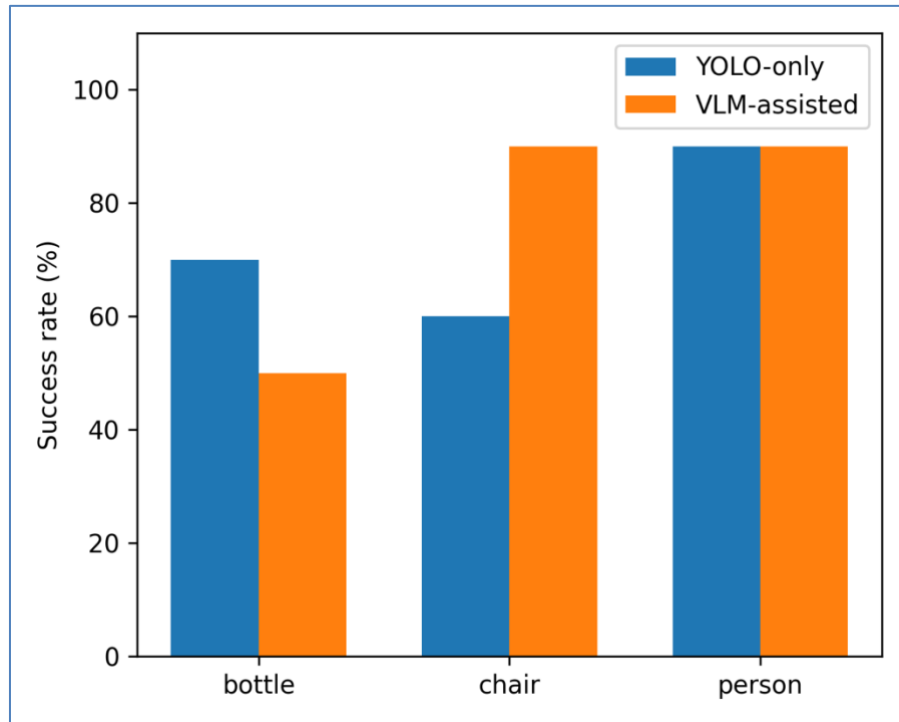
|        |                            |     |       |       |   |   |       |
|--------|----------------------------|-----|-------|-------|---|---|-------|
| person | a person facing backward   | no  | 44.21 | 11.27 | 2 | 0 | FALSE |
| person | person wearing black jeans | yes | 10.72 | 23.48 | 0 | 0 | TRUE  |

## 5.2 Overall performance

“The overall success-rate comparison in Table 5.3 and Figure 5.1 shows that the two systems behave differently across object classes.”In the YOLO-only baseline, success rates were 60% for chair, 70% for bottle, and 90% for person. In the VLM-assisted system, the success rate increased to 90% for chair, remained 90% for person, but decreased to 50% for bottle. These results indicate that the VLM layer was particularly beneficial for disambiguating chairs and maintaining robust person search, while bottle remained the hardest target for both pipelines (refer table 5.3).

**Table 5.3 Overall performance of YOLO-only and VLM-assisted commanders**

| Object class  | System       | Successful trials / 10 | Success rate (%) | Notes                                                 |
|---------------|--------------|------------------------|------------------|-------------------------------------------------------|
| <b>Chair</b>  | YOLO-only    | 6 / 10                 | 60               | Several failures due to accepting wrong or no chair   |
| <b>Chair</b>  | VLM-assisted | 9 / 10                 | 90               | Semantic layer disambiguates between multiple chairs  |
| <b>Bottle</b> | YOLO-only    | 7 / 10                 | 70               | High search effort; small crops and occlusions common |
| <b>Bottle</b> | VLM-assisted | 5 / 10                 | 50               | Conservative semantic threshold and small ROIs        |
| <b>Person</b> | YOLO-only    | 9 / 10                 | 90               | Large, visually distinctive targets                   |
| <b>Person</b> | VLM-assisted | 9 / 10                 | 90               | Similar robustness, richer description handling       |



**Figure 5.1 — Success rate by object category for YOLO-only and VLM-assisted commanders**

The chair category is the clearest example of the benefit of semantic verification. The baseline succeeded in only 6 of 10 chair trials, whereas the VLM-assisted commander succeeded in 9 of 10 chair trials. This improvement is consistent with the intended role of the VLM: instead of treating all chairs as equivalent, the semantic layer evaluates the cropped candidate object against the full description and only accepts the one that best matches the user’s language. In practical terms, when multiple chairs were present in view, the VLM layer could use natural descriptors such as color or surrounding context to pick the intended one; for example, if one chair was black and another was blue, the system could use the query description to guide the robot toward the blue chair rather than simply the first detected chair. This 30-percentage-point improvement aligns with the intended role of the VLM layer described in Section 4.5.3, where the system must semantically validate the candidate before navigation (fig 5.1).

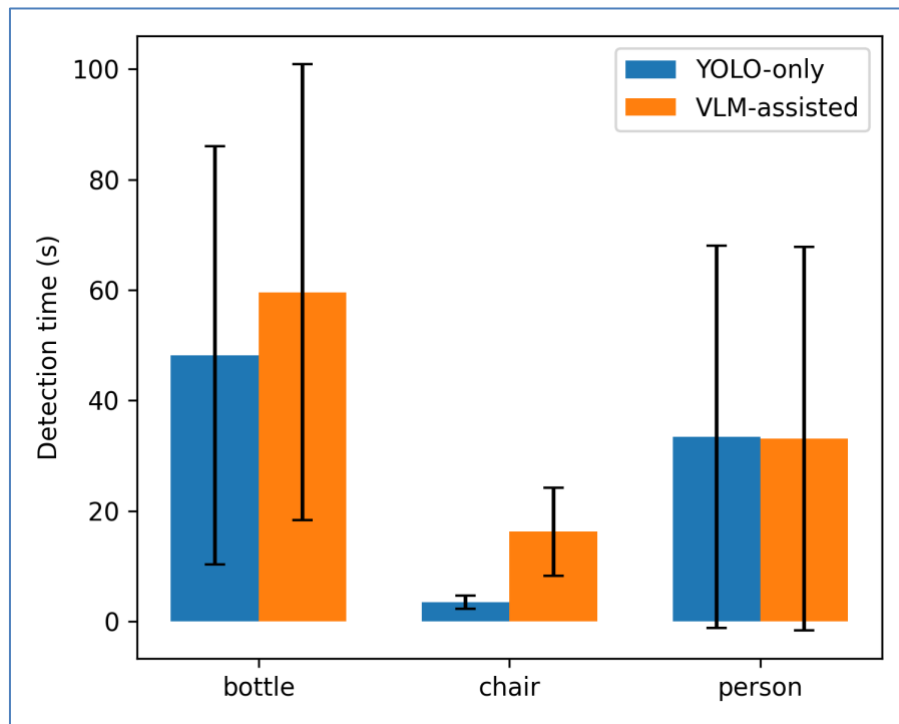
Bottle trials were the most difficult in both configurations. The baseline achieved 7 successful bottle trials out of 10, while the VLM-assisted system succeeded in only 5 out of 10. This drop appears to be associated with the combination of small object size, higher search effort, and the fact that the semantic layer applies a conservative acceptance threshold before navigation is allowed. By contrast, person trials were comparatively robust: both systems achieved 9 successes out of 10, showing that larger and more visually distinct targets are easier to detect and validate in this setup.

### 5.3 Detection time and target acquisition

The mean detection-time plot shows a clear trade-off between semantic flexibility and speed. For chairs, the YOLO-only system detected targets quickly, with most successful runs occurring between about 1.7 s and 4.7 s, whereas the VLM-assisted system took longer, with successful chair detections

ranging approximately from 7.7 s to 30.9 s. Even with this delay, the chair success rate improved substantially under VLM assistance, indicating that slower but more semantically selective confirmation was often preferable to rapid but ambiguous category-only detection.

For bottles, the mean detection time was high and highly variable in both methods. Baseline successful bottle detections ranged from about 10.25 s to 112.44 s, while VLM-assisted successful bottle detections ranged from about 19.77 s to 112.78 s. This long-tailed behavior suggests that bottles were often not visible in the first few sectors and that, even when visible, small crops and weak discriminative cues made confirmation difficult. For people, the two methods had similar mean detection times overall, though individual trials varied widely from very fast detections to long searches exceeding 100 s.



**Figure 5.2 — Mean detection time for successful trials by object category**

Reach time, once a target had been accepted, was generally more stable than detection time. This is expected because the main uncertainty lies in finding and validating the target, while the actual motion after goal publication is handled by Nav2 using a short local standoff goal. In other words, most of the variance in system performance came from target acquisition rather than from motion execution itself (refer fig 5.2).

## 5.4 Search effort

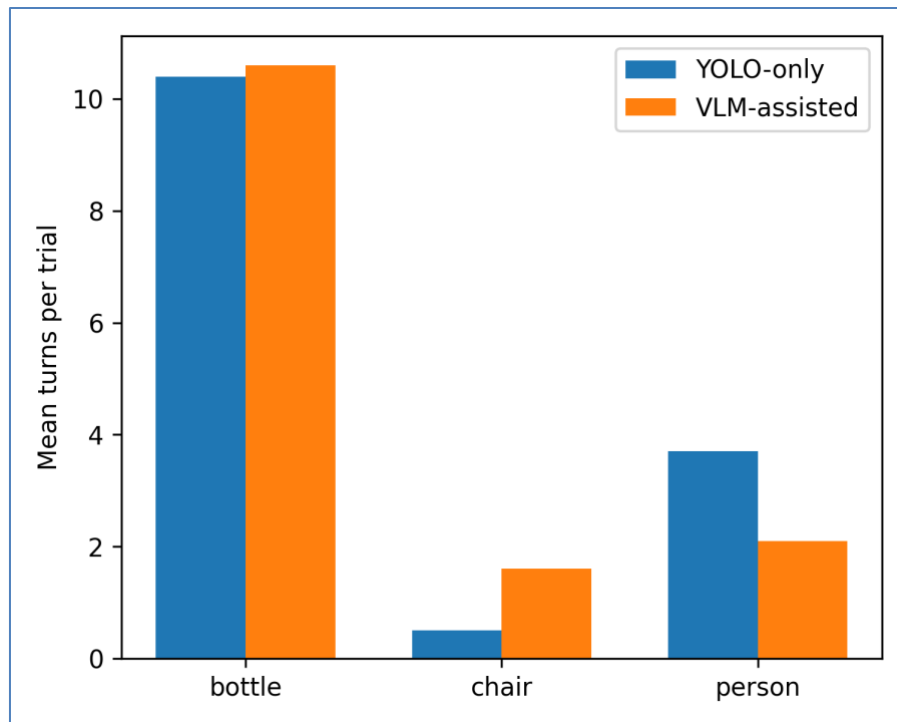


Figure 5.3 — Mean number of 45° turns per trial by object category

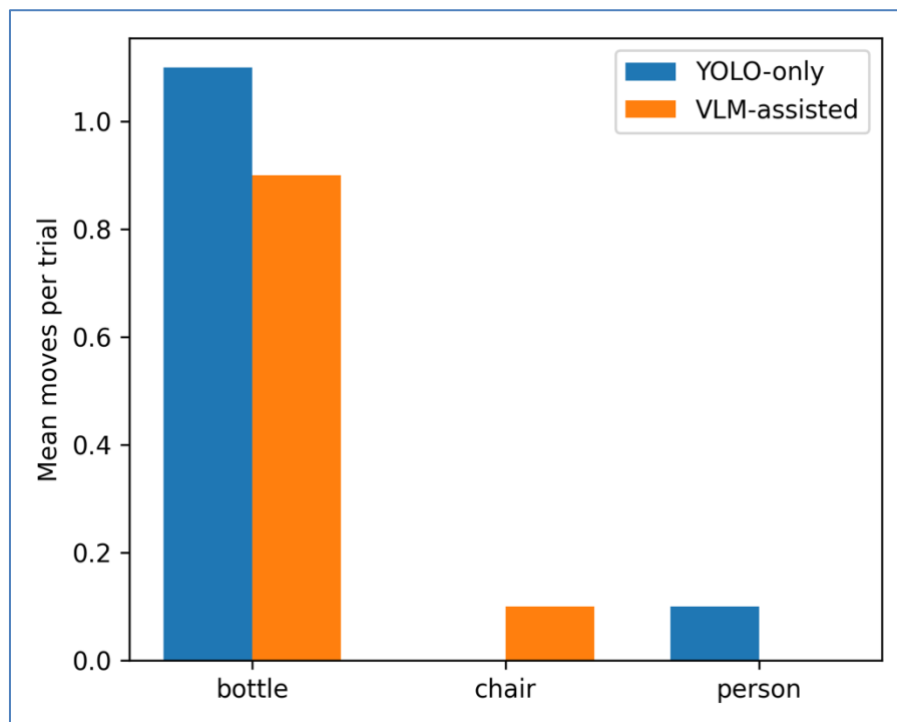


Figure 5.4 — Mean number of 1.0 m vantage-point moves per trial by object category

The search-effort plots further clarify the object-dependent difficulty of the task. Bottle trials required by far the highest number of turns and the largest number of vantage-point moves in both systems. The average number of turns per trial was above 10 for bottles in both methods, while chair and person were much lower. Similarly, bottle trials had the highest mean moves per trial, indicating that



the robot often had to abandon its current location and relocate before finding or confirming the object.

Chair trials required little search effort in the baseline and modestly more in the VLM-assisted configuration. This extra effort is consistent with a system that rejects semantically weak chair candidates until a better-matching instance is observed. Person trials showed moderate turning effort in the baseline and lower turning effort in the VLM-assisted system, which suggests that the semantic filter sometimes helped eliminate unnecessary motion by confirming the intended person once a sufficiently informative view was obtained (refer fig 5.3 & 5.4).

## 5.5 Effect of natural-language descriptions

The scatter plot relating VLM detection time to query-description length adds an important result that is specific to the semantic layer. Short generic queries such as “person,” “bottle,” or “a white chair” often produced relatively fast detections, whereas richer descriptions such as “a black chair with a bag on it,” “a green bottle with slight green cap,” or “a person sitting with a laptop” were associated with longer search times in many trials. This pattern is not surprising: longer queries encode more semantic constraints, which can improve specificity but also require the system to reject more partially matching candidates before accepting a target.

At the same time, richer descriptions also exposed a key qualitative strength of the VLM-assisted architecture. Unlike the YOLO-only baseline, which ultimately grounds the query to a single fixed class label, the VLM-assisted system keeps the full description active during target validation. As a result, it can discriminate between visually similar objects of the same class using language-grounded cues such as color, context, or relative appearance. This is visible most clearly in the chair trials, where queries referring to black, white, or grey chairs were mostly successful, and where the system could distinguish among multiple chairs in the frame based on the intended descriptive attributes (fig 5.5).

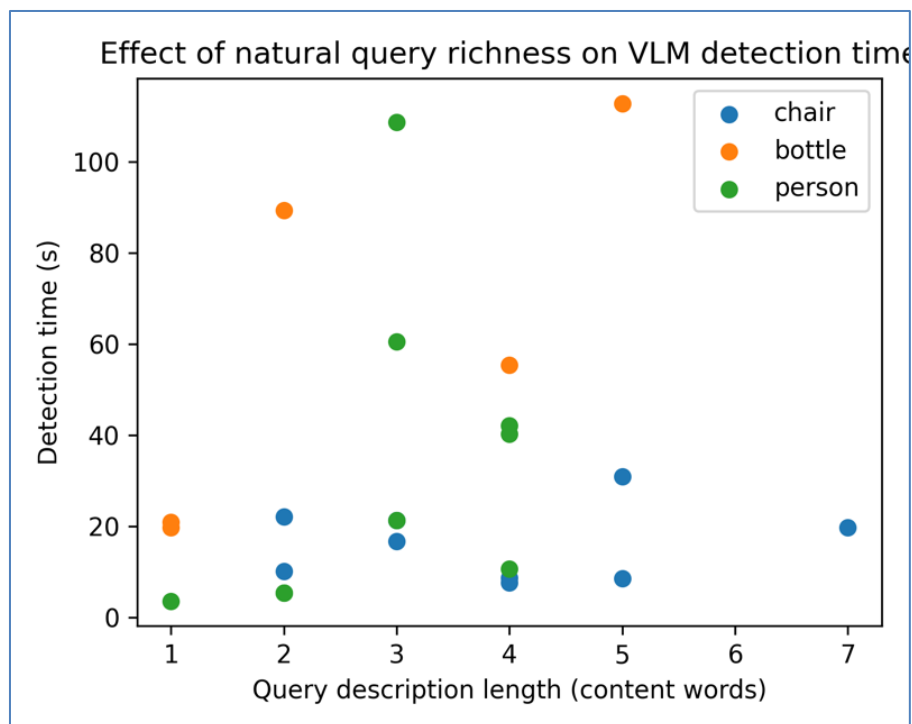


Figure 5.5 — Effect of natural-language query richness on VLM-assisted detection time

## Discussion

### 5.6 Interpretation of the findings

The results show that adding a lightweight VLM layer changes the behavior of the robot from a purely category-driven searcher into a more selective, language-grounded object finder. This selectivity improves success when the task requires discrimination between multiple instances of the same class, particularly in the chair category. The strongest evidence for this is the rise in chair success rate from 60% to 90% and the successful completion of trials with descriptions that depend on visible appearance and context, such as color, bag presence, or relative closeness to another chair.

The cost of this increased semantic specificity is additional latency. The VLM-assisted commander does not simply accept the first detected chair, bottle, or person; it must crop the ROI, send it to the remote model, parse the JSON result, compare semantic scores, and only then issue a navigation goal if the score exceeds 0.70. Because the VLM runs externally over Wi-Fi and is deliberately rate-limited through *vlm\_cooldown*, the system is slower by design than the onboard YOLO-only baseline. The detection-time graphs therefore reflect not only detector latency but the complete system-level target acquisition process, including scan dwell time, sector turns, ROI validation, and occasional network delay.

The bottle results reveal an important limitation of the current architecture. Although the VLM layer improves semantic discrimination for larger objects, it does not automatically solve the perception challenge posed by small targets. Bottles often appear as small image crops, and the code explicitly ignores detections whose width or height is below 50 pixels because such ROIs are likely to be unreliable for semantic verification. Even when a bottle crop is passed to the VLM, detailed attribute-based descriptions can make acceptance harder, especially if the crop is small, partially visible, or weakly illuminated. This explains why bottle trials often required many turns and moves and why several descriptive bottle queries failed despite long search times.

The person category illustrates a more balanced outcome. Both systems achieved high success, but the VLM-assisted system retained the ability to handle richer descriptions such as clothing color, pose, or context (“a person sitting with a laptop”), which are not semantically actionable in the baseline. The one VLM-assisted failure marked as a false positive also shows that semantic reasoning is helpful but not perfect; the VLM can still be overconfident in some cases, which justifies the conservative thresholding and the decision to use it as an advisor rather than as a controller.

Overall, the experiments support the main thesis claim that a small external VLM can usefully augment a classical YOLO + Nav2 robot without replacing the reliable geometric navigation stack. The improvement is most evident when the task involves natural-language disambiguation among same-class objects, such as choosing one chair over another based on color or contextual cues. At the same time, the experiments also show the current limits of this approach: the added semantic power comes with extra target-acquisition delay, and the approach remains constrained by the quality of the initial detector proposals, especially for small objects like bottles.

## Chapter 6: Conclusion

This study has shown how a lightweight vision-language model can be used together with a traditional YOLO + Nav2 navigation stack for natural-language object-goal navigation on a mobile robot with constrained hardware. Specifically, the focus was on employing VLM as a semantic component, not an end-to-end controller, to preserve the geometric planning and obstacle avoidance functionality inherent in Nav2 and give the robot an extra capacity to process natural language descriptions of the objects.

The main research questions posed in Chapter 2 included the ways in which natural-language instructions can influence the search strategy, whether an off-the-shelf tiny VLM can offer adequate semantic guidance, the techniques for combining its output with YOLO detections and Nav2, and how effective this combination is going to be. These issues are resolved by a sector-based search state machine and ROI-based semantic verification – the input from a user in the form of a query remains unprocessed (i.e., in its original natural language format), the detected objects are scored using VLM, and the accepted targets are translated into goals in Nav2 coordinates.

In terms of the implementation, the thesis provided a modular structure in which the VLM worked as an external service having defined timing and resource constraints. Bounding boxes were generated by YOLO or OAK-D spatial detectors, cropped and filtered by the VLM commander based on minimal size and cool-down requirements, while the navigation commands were issued by the Nav2 stack with respect to a set standoff distance for short-range navigation. The algorithm responsible for semantic validation was written in `evaluate_detections()`, formally expressed by Algorithm 4.2, and involved a confidence level of 0.70 and the best-score policy to identify at most one object in each image frame.

Experimentation with a differential drive robot in an indoor lab environment evaluated the baseline approach with the only YOLO model against the novel approach using the VLM-based commander with regard to three classes: chair, bottle, and person. In 30 trials for both methods, the use of the VLM-based approach improved the success of chair tasks from 6/10 to 9/10 trials, kept high success in person-related tasks at 9/10 trials, but diminished bottle task success from 7/10 to 5/10 trials. The obtained results demonstrate the primary hypothesis about the improvement in performance on tasks involving disambiguation within classes achieved by a minor VLM due to color or other aspects, yet semantically strict verification negatively affects smaller objects' detection and recognition.

The evaluation of detection time and effort made it clear about the implications of introducing the semantic layer to the task of object recognition. As for the detection of chairs, the VLM-based system took more time than YOLO alone to recognize a chair but succeeded due to dismissing ambiguous examples in favor of finding a more matching one. As far as bottles are concerned, both approaches had rather long and unpredictable times to detect this object, while the bottle test involved the most turns of 45° and relocating of 1 m – indicating that it is hard to detect and recognize such small objects.

These results indicate that the architecture effectively enables the robot to change from being a purely category-based search robot into a more selective language-driven object finder. The most

compelling quantitative results include better performance regarding the chair task, as well as performing tasks based on perceptual cues such as finding "a black chair with a bag" and "a white chair without a bag". However, the experiments emphasize that the VLM needs to be regarded as a probabilistic semantic sensor, which can make mistakes and suffers from limitations when it comes to the quality of image crops available.

These restrictions define the principal restrictions in this work. First, the robot operates in relatively small indoor spaces, takes account of one object at a time, and assumes good localisation provided by Nav2. Furthermore, the VLM is computed on an external computer using the Wi-Fi connection, leading to extra delays and necessitating cooldown and increased sector times to account for that. As a consequence, VLM-assisted commander is intentionally slow when compared to the baseline that uses only YOLO. Second, the fixed size of at least 50 pixels of the cropped image has been chosen because of the noise issue; hence, it has an impact on small objects like bottles.

Based on these limitations, there are a few research paths we can follow. One path is the development of a multi-resolution or multi-scale object detection approach that varies the size of the minimal ROI based on the type and distance of the object being detected. Another is to explore how we can lower semantic latency through the onboard deployment of quantized or distilled versions of the VLM, or even using cached VLM responses for crops that have been seen before to decrease reliance on Wi-Fi and the external computer. The third path would be to take the single object, single command framework and scale it to allow for multi-step plans that include richer semantic memory or even uncertainty-aware planning that allows the robot to build a long-term understanding of where objects have been seen and thus where to look next.

## Chapter 7: References

1. Chaplot, D. S., Gandhi, D., Gupta, S., et al. “Learning to Map for Active Semantic Goal Navigation.” *International Conference on Learning Representations (ICLR)*, 2022.
2. Razavi, M. “Semantic Object-Goal Navigation on a Quadruped Robot in Known Indoor Environments Under Tight Computational Constraints.” Master’s thesis, Aalborg University, 2025.
3. Ramakrishnan, S. K., et al. “HM3D-OVON: A Benchmark for Open-Vocabulary Object-Goal Navigation in Large-Scale 3D Environments.” arXiv preprint, 2024.
4. Wang, T., Fan, J., Zheng, P., Yan, R., Wang, L. “Vision-Language Models Revolutionize Mobile Robot Navigation in Human-Centric Smart Manufacturing.” *Engineering*, 2026.
5. Huang, J., Abbeel, P., Pathak, D. “VLMaps: Vision-Language Maps for Zero-Shot Robot Navigation.” arXiv preprint, 2023.
6. Bajpai, N., et al. “Uncertainty-Aware Active Perception for Open-Vocabulary Object-Goal Navigation.” arXiv preprint, 2025.
7. Xue, F., et al. “Integrating Large Language Models for Intuitive Robot Navigation.” arXiv preprint, 2024.
8. Zhang, Y., et al. “Unblocking the Path: VLM-Assisted Robot Navigation in Indoor Environments.” arXiv preprint, 2025.
9. Li, X., et al. “A Vision-Language Framework for Assistive Home Robotics.” *European Conference on Mobile Robotics (ECMR)*, 2024.
10. Sun, Y., et al. “OLiVia-Nav: An Online Lifelong Vision-Language Approach for Mobile Robot Social Navigation.” arXiv:2409.13675, 2024.
11. Singh, A., et al. “VL-Nav: A Neuro-Symbolic Approach for Reasoning-Based Vision-Language Navigation.” Technical report, SAIR Lab, 2026.
12. Redmon, J., et al. “You Only Look Once: Unified, Real-Time Object Detection.” *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. (YOLO baseline for object detection.)
13. Macenski, S., Foote, T., Gerkey, B., Lalancette, C., Woodall, W. “Robot Navigation 2: Design and Architecture.” *IEEE Robotics and Automation Magazine*, 2020. (Nav2 stack for classical navigation.)

## Appendix A

**Table A.1 — Fields in the VLM JSON response**

| Field             | Type   | Description                                                                                          |
|-------------------|--------|------------------------------------------------------------------------------------------------------|
| match             | bool   | True if the VLM judges that the crop matches the user’s description within the configured threshold. |
| score             | float  | Confidence score in the range [0, 1] used for thresholding with vlm_threshold.                       |
| reason            | string | Short natural-language justification indicating why the model accepted or rejected the crop.         |
| object_type       | string | Noun phrase describing the object that best matches the crop (for logging and analysis).             |
| dir<br>(optional) | string | Coarse left/center/right location cue used by the search state machine when available.               |

**Table A.2 — Main ROS 2 topics in the system**

| Topic name              | Message type                                | Publisher node      | Subscriber node(s)            | Purpose                              |
|-------------------------|---------------------------------------------|---------------------|-------------------------------|--------------------------------------|
| /camera/color/image_raw | sensor_msgs/msg/Image                       | OAK-D camera driver | YOLO commander, VLM commander | RGB input for detection and cropping |
| /yolo_detections        | custom or vision_msgs/msg/Detection2D Array | YOLO commander      | VLM commander                 | 2D detections used to generate ROIs  |
| /spatial_detections     | DepthAI spatial detection msg               | OAK spatial node    | VLM commander                 | 3D positions of detected objects     |
| /cmd_vel                | geometry_msgs/msg/Twist                     | VLM commander, Nav2 | Differential drive base       | Low-level velocity commands          |

| Topic name   | Message type                                | Publisher node | Subscriber node(s) | Purpose                               |
|--------------|---------------------------------------------|----------------|--------------------|---------------------------------------|
| /goal_pose   | geometry_msgs/msg/PoseStamped               | VLM commander  | Nav2               | Local goal for object-goal navigation |
| /nav2/status | nav2_msgs/msg/BehaviorTreeLog or status msg | Nav2           | VLM commander      | Monitoring navigation outcome         |

**Table A.3 — Important semantic and search parameters**

| Parameter         | Value                    | Description                                                                                         |
|-------------------|--------------------------|-----------------------------------------------------------------------------------------------------|
| vlm_threshold     | 0.70                     | Minimum allowed VLM confidence for accepting a detection.                                           |
| vlm_cooldown      | 2–3 frames (approx. 2 s) | Cooldown interval during which new crops are not sent to the VLM to limit query rate.               |
| min_crop_size_px  | 50                       | Minimum width or height in pixels for a detection to be considered for semantic verification.       |
| sector_dwell_yolo | 5.0 s                    | Time spent scanning each sector in the YOLO-only commander.                                         |
| sector_dwell_vlm  | 10.0 s                   | Time spent scanning each sector in the VLM-assisted commander to accommodate semantic verification. |
| standoff_distance | 0.5 m                    | Distance from the detected object at which the Nav2 goal is placed.                                 |